

Marcelo Claro Laranjeira

**OPENCODE ECOSYSTEM: ARQUITETURA,
OPERAÇÃO E EVOLUÇÃO DE UMA
PLATAFORMA MULTIAGENTE AUTÔNOMA
PARA ENGENHARIA DE SOFTWARE E
PRODUÇÃO DE CONHECIMENTO CIENTÍFICO
VERIFICÁVEL**

Fortaleza – Ceará

2026

Marcelo Claro Laranjeira

**OPENCODE ECOSYSTEM: ARQUITETURA, OPERAÇÃO
E EVOLUÇÃO DE UMA PLATAFORMA MULTIAGENTE
AUTÔNOMA PARA ENGENHARIA DE SOFTWARE E
PRODUÇÃO DE CONHECIMENTO CIENTÍFICO
VERIFICÁVEL**

Tese apresentada como requisito parcial para
obtenção do título de Doutor em Engenharia de
Software.

Orientador: [ORIENTADOR]
Coorientador: [COORIENTADOR]

Fortaleza – Ceará
2026

Ficha Catalográfica

LARANJEIRA, Marcelo Claro. OpenCode Ecosystem: Arquitetura, Operação e Evolução de uma Plataforma Multiagente Autônoma para Engenharia de Software e Produção de Conhecimento Científico Verificável. Fortaleza, 2026. 74 p. Tese (Doutorado em Engenharia de Software). 1. Engenharia de Software. 2. Sistemas Multiagente. 3. Model Context Protocol. 4. Verificação Formal. 5. Ecossistema OpenCode. I. Título.

Agradecimentos

À comunidade de software livre que produziu as infraestruturas fundamentais sobre as quais esta pesquisa se ergue: o OpenCode CLI da AnomalyCo, o MiroFish e o BettaFish de 666ghj, o DeerFlow da ByteDance, e os milhares de contribuidores anônimos cujas bibliotecas, frameworks e protocolos formam o substrato técnico do ecossistema aqui documentado.

Aos revisores anônimos que examinaram versões preliminares deste manuscrito e cujas críticas substantivas – por vezes desconfortáveis, sempre necessárias – elevaram significativamente a qualidade do texto final.

*“The act of describing a program in unambiguous detail
and the act of programming are one and the same.”*

— K. E. Iverson

RESUMO

Esta tese apresenta o projeto, a implementação, a documentação e a validação do Ecossistema OpenCode – uma plataforma multiagente autônoma para engenharia de software e produção de conhecimento científico verificável. O problema central investigado é como arquitetar sistemas de software onde 125 agentes de inteligência artificial operam colaborativamente, mantendo simultaneamente verificabilidade formal de cada afirmação produzida, rastreabilidade integral de cada decisão arquitetural e qualidade de engenharia compatível com padrões Qualis A1. A solução proposta organiza-se em uma arquitetura de três camadas estritas – Infraestrutura (46 Model Context Protocols), Domínio (150 skills em 13 categorias) e Orquestração (125 agentes tipados) – governada por seis princípios arquiteturais explícitos derivados da engenharia de software clássica e validados empiricamente ao longo de 13 ciclos evolutivos. A camada de infraestrutura implementa o protocolo MCP como barramento de interoperabilidade, permitindo que qualquer agente acesse qualquer ferramenta externa através de interface padronizada, sem código de integração customizado. A camada de domínio organiza conhecimento especializado em unidades atômicas (skills) com mecanismo de progressive disclosure para eficiência de contexto. A camada de orquestração coordena agentes através de delegação de tarefas, comunicação assíncrona via sistema de arquivos (File IPC) e debate multiagente moderado por LLM no Agent Forum. A verificação formal é implementada pelo pipeline Cora-Debate, onde 7 verificadores simbólicos independentes (V1-V7) operam em paralelo sobre cada afirmação, produzindo um vetor de scores de confiança que alimenta um mecanismo de consenso adaptativo baseado em Q-Score UCB1 e calibração Platt. O desenvolvimento de software é governado pelo pipeline SDD+TDD, onde especificações formais atuam simultaneamente como contratos entre stakeholders humanos e agentes, como contexto operacional para os próprios agentes e como oráculo para validação automatizada. A auditabilidade é garantida por sistema de logging imutável (JSONL append-only) com hashing SHA-256 encadeado formando uma blockchain de auditoria, e pelo protocolo TSAC (Textual Source Analysis Credibility) que exige cinco componentes verificáveis por citação. A validação empírica através de 200 casos de teste no benchmark CORA-Eval demonstrou 100% de aprovação em 11 dimensões de raciocínio científico, com F1-scores individuais dos verificadores entre 0.90 e 0.99. A trajetória de 13 ciclos evolutivos (score 85→100, +17.6%) evidencia melhoria monotônica com aceleração na terceira geração, consistente com o fenômeno de J-curve de refatoração documentado na literatura de engenharia de software. As contribuições incluem: (i) arquitetura de referência MCP-first formalmente especificada; (ii) taxonomia de 212 tipos de raciocínio com validação empírica; (iii) framework de verificação formal multi-paradigma integrando SMT, álgebra simbólica e lógica relacional; (iv) metodologia SDD+TDD para engenharia de software com agentes de IA; (v) sistema de auditoria caixa branca com rastreabilidade criptográfica.

Palavras-chave: Engenharia de Software. Sistemas Multiagente. Model Context Protocol. Verificação Formal. Arquitetura de Software. Ecossistema OpenCode.

ABSTRACT

This thesis presents the design, implementation, documentation, and validation of the OpenCode Ecosystem – an autonomous multi-agent platform for software engineering and verifiable scientific knowledge production. The central problem investigated is how to architect software systems where 125 AI agents operate collaboratively while simultaneously maintaining formal verifiability of every produced claim, complete traceability of every architectural decision, and engineering quality compatible with Qualis A1 standards. The proposed solution is organized into a strictly layered three-tier architecture – Infrastructure (46 Model Context Protocols), Domain (150 skills across 13 categories), and Orchestration (125 typed agents) – governed by six explicit architectural principles derived from classical software engineering and empirically validated over 13 evolutionary cycles. The infrastructure layer implements the MCP protocol as an interoperability bus, enabling any agent to access any external tool through a standardized interface without custom integration code. The domain layer organizes specialized knowledge into atomic units (skills) with a progressive disclosure mechanism for context efficiency. The orchestration layer coordinates agents through task delegation, asynchronous file-system-based communication (File IPC), and LLM-moderated multi-agent debate in the Agent Forum. Formal verification is implemented by the Cora-Debate pipeline, where 7 independent symbolic verifiers (V1-V7) operate in parallel on each claim, producing a vector of confidence scores that feeds an adaptive consensus mechanism based on Q-Score UCB1 and Platt calibration. Software development is governed by the SDD+TDD pipeline, where formal specifications simultaneously serve as contracts between human stakeholders and agents, as operational context for the agents themselves, and as oracles for automated validation. Auditability is ensured by an immutable logging system (append-only JSONL) with chained SHA-256 hashing forming an audit blockchain, and by the TSAC (Textual Source Analysis Credibility) protocol requiring five verifiable components per citation. Empirical validation through 200 test cases in the CORA-Eval benchmark demonstrated 100% approval across 11 scientific reasoning dimensions, with individual verifier F1-scores between 0.90 and 0.99. The trajectory of 13 evolutionary cycles (score 85→100, +17.6%) demonstrates monotonic improvement with acceleration in the third generation, consistent with the refactoring J-curve phenomenon documented in the software engineering literature. Contributions include: (i) formally specified MCP-first reference architecture; (ii) taxonomy of 212 reasoning types with empirical validation; (iii) multi-paradigm formal verification framework integrating SMT, symbolic algebra, and relational logic; (iv) SDD+TDD methodology for AI-agent software engineering; (v) white-box auditing system with cryptographic traceability.

Keywords: Software Engineering. Multi-Agent Systems. Model Context Protocol. Formal Verification. Software Architecture. OpenCode Ecosystem.

Lista de ilustrações

Figura 1 – Ciclo evolutivo PLAN-ACT-REFLECT-EXTRACT-EVOLVE. A cada volta completa (iteração t), o ecossistema analisa métricas, implementa mudanças, avalia o impacto, extrai padrões de conhecimento e os incorpora à arquitetura. O ciclo se repete indefinidamente ($t \rightarrow t + 1 \rightarrow t + 2 \dots$).	31
Figura 2 – Arquitetura em três camadas do Ecossistema OpenCode.	38
Figura 3 – Fluxo de interação MCP: (1) agente decide; (2) Host despacha em paralelo; (3) servidores respondem; (4) tudo é registrado no InteractionLogger; (5) resultados voltam ao agente.	40
Figura 4 – Mecanismos de comunicação inter-agente: Task Delegation, File IPC e Agent Forum.	44
Figura 5 – Pipeline Cora-Debate: 7 verificadores paralelos $\rightarrow$ consenso UCB1 $\rightarrow$ calibração Platt.	52
Figura 6 – Pipeline SDD+TDD: 5 estágios sequenciais com ciclos de retroalimentação.	55

Lista de tabelas

Tabela 1 – Catálogo completo de servidores MCP do OpenCode	39
Tabela 2 – Distribuição completa das 150 skills por categoria	43
Tabela 3 – Métricas de qualidade de código e thresholds	58
Tabela 4 – Os 13 ciclos evolutivos do OpenCode	59
Tabela 5 – Critérios de avaliação Qualis A1 com pesos	62
Tabela 6 – Métricas de evolutibilidade ao longo de 13 ciclos	66
Tabela 7 – Resultados completos do CORA-Eval Benchmark	67

Sumário

Agradecimentos	4
Lista de ilustrações	8
Lista de tabelas	8
Sumário	9
1 Introdução	13
1.1 Contextualização e Motivação	13
1.2 Formulação do Problema de Pesquisa	14
1.3 Hipóteses de Pesquisa	15
1.4 Objetivos	16
1.4.1 Objetivo Geral	16
1.4.2 Objetivos Específicos	16
1.5 Delimitação do Escopo	17
1.6 Contribuições Esperadas	18
1.7 Estrutura da Tese	19
2 Fundamentação Teórica	20
2.1 Sistemas Multiagente: Dos Fundamentos aos LLM-MAS	20
2.1.1 Definição Formal de Sistema Multiagente	20
2.1.2 As Quatro Gerações de Arquiteturas de Agentes	21
2.1.3 Desafios de Engenharia em LLM-MAS	22
2.2 Model Context Protocol: O Barramento de Interoperabilidade	23
2.2.1 O Problema da Integração LLM-Ferramenta	23
2.2.2 As Quatro Primitivas do MCP	23
2.2.3 Topologia e Transportes	24
2.2.4 Como o OpenCode Utiliza o MCP na Prática	24
2.2.5 Propriedades Formais da Arquitetura MCP	25
2.2.6 Limitações do MCP e Estratégias de Contorno	26
2.3 Engenharia de Software com Agentes Inteligentes	26
2.3.1 O Paradigma SDD (Spec-Driven Development)	26
2.3.2 O Ciclo TDD e sua Extensão para Agentes IA	27
2.3.3 O Pipeline SDD+TDD Integrado	28
2.4 Motores de Raciocínio Formal	29
2.4.1 Fundamentos e Justificativa	29
2.4.2 Z3 Theorem Prover: Verificação SMT	29
2.4.3 SymPy: Álgebra Simbólica	29
2.4.4 miniKanren: Programação Lógica Relacional	30
2.4.5 Critical Engine: Detecção de Falácias	30

2.5	Ecosistemas de Software Evolutivos	30
2.5.1	As Leis de Lehman e a Evolução de Software	30
2.5.2	O Ciclo Evolutivo Autônomo	31
3	Estado da Arte e Revisão Sistemática	32
3.1	Metodologia da Revisão	32
3.2	Análise por Eixo Temático	33
3.2.1	Eixo 1: LLM-MAS para Engenharia de Software	33
3.2.2	Eixo 2: Model Context Protocol	34
3.2.3	Eixo 3: Verificação Formal de Código Gerado por IA	34
3.2.4	Eixo 4: Integridade Acadêmica na Era da IA	35
3.2.5	Eixo 5: Arquitetura de Software e Gestão de Conhecimento	35
3.3	Síntese Crítica e Posicionamento da Tese	35
4	Arquitetura do Ecosistema OpenCode	37
4.1	Visão Arquitetural Global	37
4.1.1	Os Seis Princípios Arquiteturais	38
4.2	Camada de Infraestrutura: O Barramento MCP	39
4.2.1	Catálogo de Servidores MCP	39
4.2.2	Protocolo de Comunicação Detalhado	40
4.2.3	Exemplo de Interação MCP: Instrumentada e Auditada	40
4.3	Camada de Domínio: O Sistema de Skills	41
4.3.1	Anatomia de uma Skill	41
4.3.2	Mecanismo de Carregamento e Progressive Disclosure	42
4.3.3	Taxonomia das 150 Skills	43
4.4	Camada de Orquestração: Agentes e Coordenação	43
4.4.1	O Modelo de Agente do OpenCode	43
4.4.2	Mecanismos de Comunicação Inter-Agente	44
4.5	Componentes Transversais	45
4.5.1	Sistema de Permissões	45
4.5.2	DecisionNode	45
4.5.3	Plugin Manus-Evolve	45
5	Motor de Raciocínio Multi-Paradigma	46
5.1	Fundamentação Epistemológica	46
5.2	A Taxonomia de Raciocínios: Estrutura e Gênese	46
5.2.1	As Seis Famílias Epistemológicas	47
5.3	Arquitetura de Paralelismo do Reasoning Orchestrator	48
5.3.1	Paralelismo Intra-Fase	48
5.3.2	Paralelismo Inter-Fase	48
5.3.3	Paralelismo Multi-Caminho (Self-Consistency)	49
5.4	Os Quatro Motores de Raciocínio Formal	49

5.4.1	Z3 Theorem Prover: Operação e Integração	49
5.4.2	SymPy: Verificação Algébrica Automatizada	49
5.4.3	miniKanren: Verificação Relacional	50
5.4.4	Critical Engine: Detecção de Falácias	50
5.5	Seleção Adaptativa de Estratégias de Raciocínio	50
6	Pipeline de Verificação Formal Cora-Debate	51
6.1	Arquitetura do Pipeline	51
6.2	Especificação Técnica dos Sete Verificadores	52
6.2.1	V1 – Análise Dimensional	52
6.2.2	V2 – Verificação Algébrica	52
6.2.3	V3 – Geração de Contraexemplos	52
6.2.4	V4 – Validação Estatística	53
6.2.5	V5 – Verificação Numérica	53
6.2.6	V6 – Validação de EDOs e PDEs	53
6.2.7	V7 – Consistência Lógica	53
6.3	Mecanismo de Consenso Adaptativo	53
6.3.1	Q-Score UCB1	53
6.3.2	Calibração Platt	54
6.4	Validação Empírica: Benchmark CORA-Eval	54
7	Engenharia de Software com Agentes Inteligentes	55
7.1	O Problema do Desenvolvimento com Agentes de IA	55
7.2	O Pipeline em Cinco Estágios	55
7.2.1	Estágio 1 – SPECIFY: Da Ideia à Especificação Formal	56
7.2.2	Estágio 2 – DERIVE: Geração Automática de Testes	56
7.2.3	Estágio 3 – IMPLEMENT: Desenvolvimento Guiado por Testes	56
7.2.4	Estágio 4 – VERIFY: Verificação Formal	57
7.2.5	Estágio 5 – RECORD: Documentação de Decisões	57
7.3	Métricas de Qualidade de Código	57
7.4	Ciclo de Evolução Contínua	58
8	Sistema de Auditoria Acadêmica Caixa Branca	60
8.1	Os Três Pilares da Auditabilidade	60
8.1.1	InteractionLogger: Especificação Técnica	60
8.1.2	Protocolo TSAC: Cinco Componentes por Citação	61
8.1.3	Métricas de Qualidade Qualis A1	61
9	Metodologia de Pesquisa	63
9.1	Design Metodológico	63
9.1.1	Estudo de Caso Único Longitudinal	63
9.1.2	Pesquisa-Ação (Action Research)	63
9.1.3	Instrumentos de Coleta de Dados	63

9.2	Ameaças à Validade e Estratégias de Mitigação	64
9.2.1	Validade Interna	64
9.2.2	Validade Externa	64
9.2.3	Validade de Constructo	64
9.2.4	Validade de Conclusão	65
10	Resultados e Discussão	66
10.1	Arquitetura e Evolutibilidade	66
10.2	Desempenho do Cora-Debate	66
10.3	Pipeline SDD+TDD	67
10.4	Evolução Temporal da Qualidade	67
10.5	Consumo de Recursos	68
11	Discussão e Implicações	69
11.1	Implicações para a Engenharia de Software	69
11.1.1	O Paradigma MCP-First	69
11.1.2	Especificação como Infraestrutura, não como Documentação	69
11.1.3	Agentes como Stakeholders de Primeira Classe	69
11.2	Implicações para Ecossistemas de Software	70
11.2.1	Ecossistemas vs. Aplicações: Uma Distinção Qualitativa	70
11.2.2	Sustentabilidade de Ecossistemas Open-Source com IA	70
11.3	Limitações e Trabalhos Futuros	70
12	Conclusão	71
	Referências	73

1 Introdução

1.1 Contextualização e Motivação

O campo da engenharia de software atravessa uma transformação estrutural cujo alcance somente agora começa a ser compreendido. Durante cinco décadas, a disciplina desenvolveu-se sobre o pressuposto fundamental de que software é produzido por programadores humanos – seres que raciocinam sobre requisitos, decompõem problemas, escrevem código, testam implementações e documentam decisões. Este pressuposto organizou nossos métodos (cascata, ágil, DevOps), nossas métricas (linhas de código, cobertura de testes, complexidade ciclomática) e nossos artefatos (especificações, código-fonte, testes, documentação).

A emergência dos Large Language Models (LLMs) rompeu este pressuposto de maneira irreversível. Modelos como GPT-4, Claude e DeepSeek demonstram capacidade de gerar código funcional a partir de descrições em linguagem natural, mas sua contribuição mais profunda não está na geração de código em si – está na capacidade de participar do **raciocínio de engenharia**: analisar requisitos, propor arquiteturas, identificar trade-offs, gerar testes, revisar código, documentar decisões. Esta capacidade transforma os LLMs de ferramentas de produtividade individual em **agentes de engenharia de software** – entidades que participam do processo de desenvolvimento não como instrumentos passivos, mas como colaboradores ativos com capacidades de raciocínio próprias.

O Model Context Protocol (MCP), introduzido pela Anthropic em novembro de 2024, catalisou esta transformação ao estabelecer um padrão aberto para interoperabilidade entre LLMs e ferramentas externas ([Ayyagari, 2025](#)). O MCP resolve o problema fundamental de conectar o raciocínio abstrato de um modelo de linguagem ao mundo concreto das ferramentas de software: sistemas de arquivos, bancos de dados, APIs, navegadores, mecanismos de busca. Ao padronizar esta interface, o MCP permitiu que a integração entre LLMs e ferramentas deixasse de ser um problema de engenharia caso a caso para tornar-se uma questão de configuração declarativa ([Lei; Xu; Liang, 2026](#)).

O Ecossistema OpenCode emerge precisamente na interseção destas duas forças transformadoras: a emergência de agentes de engenharia de software baseados em LLM e a disponibilidade de um protocolo aberto de interoperabilidade. O OpenCode não é meramente um assistente de codificação – é uma plataforma onde 125 agentes especializados, operando sobre 46 servidores MCP, com acesso a 150 módulos de conhecimento (skills), colaboram para produzir software e conhecimento científico com verificabilidade formal, rastreabilidade integral e qualidade de engenharia.

A motivação para esta pesquisa decorre de três observações convergentes, cada uma documentada na literatura e confirmada pela experiência operacional com o ecossistema. Primeira, a

pesquisa em sistemas multiagente para engenharia de software avançou significativamente em componentes isolados – revisões sistemáticas documentam 114 estudos sobre LLM-MAS para código ([RASHEED et al., 2026](#)), frameworks de orquestração como CodeAir demonstram a viabilidade de arquiteturas multiagente para desenvolvimento ([RANGEL, 2026](#)), e trabalhos sobre verificação formal estabelecem métodos para validação de código gerado por IA ([ANTERO et al., 2024](#)) – mas a literatura carece de descrições de plataformas que integrem todos estes componentes em um ecossistema coeso e operacional. Segunda, a grande maioria dos estudos avalia sistemas em experimentos pontuais de curta duração, oferecendo evidência limitada sobre o comportamento destes sistemas em operação prolongada, sua capacidade de evolução autônoma e sua resiliência a falhas em produção. Terceira, e mais determinante, a questão da **verificabilidade** – como um observador externo pode confirmar, de maneira independente e reproduzível, que o software e o conhecimento produzidos por um ecossistema multiagente atendem a padrões de qualidade e correção – permanece sem resposta definitiva, com implicações significativas para a adoção destes sistemas em contextos regulados (acadêmico, financeiro, médico, jurídico).

1.2 Formulação do Problema de Pesquisa

O problema central investigado nesta tese pode ser enunciado de forma concisa: **como projetar, implementar e validar uma arquitetura de software para ecossistemas multiagente que suporte simultaneamente evolução autônoma, verificabilidade formal, rastreabilidade integral e qualidade de engenharia?**

Este enunciado decompõe-se em cinco questões de pesquisa inter-relacionadas, cada uma abordando uma dimensão específica do problema:

QP1 – Arquitetura:

Que organização estrutural – em termos de camadas, componentes, interfaces e protocolos de comunicação – permite que um ecossistema multiagente evolua autonomamente (adicionando, modificando e depreciando componentes) sem degradação arquitetural, mantendo desacoplamento entre camadas e garantindo que a falha de um componente não se propague para o sistema como um todo?

QP2 – Interoperabilidade:

De que forma o Model Context Protocol viabiliza a integração escalável e desacoplada entre agentes heterogêneos e ferramentas externas? Quais são as limitações intrínsecas do protocolo – em termos de descoberta de serviços, composição de operações, versionamento e segurança – e como estas limitações são endereçadas (ou contornadas) na arquitetura do OpenCode?

QP3 – Raciocínio:

Como integrar múltiplos paradigmas de raciocínio – lógico-formal (dedução, indução, abdução), dialético (tese-antítese-síntese, debate), estatístico (testes de hipótese,

correlação, modelagem), estratégico (teoria dos jogos, decisão sob incerteza) e criativo (analogia, reenquadramento, design thinking) – em um pipeline computacional coerente que produza resultados verificáveis, e como orquestrar a interação entre estes paradigmas de forma que se complementem em vez de conflitarem?

QP4 – Qualidade de Engenharia:

Que mecanismos da engenharia de software tradicional – especificação formal, desenvolvimento dirigido por testes, integração contínua, registro de decisões arquiteturais – permanecem necessários e eficazes quando os agentes produtores de código não são humanos mas sistemas de IA? Como estes mecanismos precisam ser adaptados para o novo paradigma, e que novos mecanismos (ex: verificadores simbólicos, audit trail imutável) tornam-se necessários?

QP5 – Auditabilidade:

Como implementar um sistema de auditoria que permita a um observador externo verificar, de maneira independente e reproduzível, a correção e a integridade de cada afirmação produzida pelo ecossistema? Que propriedades criptográficas, estruturais e processuais são necessárias para que esta auditoria seja considerada completa e confiável?

1.3 Hipóteses de Pesquisa

Com base na revisão da literatura apresentada no Capítulo 3 e na experiência operacional acumulada ao longo de 13 ciclos evolutivos do ecossistema, formulamos quatro hipóteses de pesquisa que orientam a investigação empírica desta tese:

H1 – Hipótese da Arquitetura em Camadas:

Uma arquitetura estritamente estratificada em três camadas – Infraestrutura (MCPs), Domínio (Skills) e Orquestração (Agentes) – com interfaces padronizadas entre camadas e componentes transversais para preocupações ortogonais (logging, permissões, evolução), produz maior evolutibilidade que arquiteturas monolíticas ou fracamente estratificadas. A evolutibilidade é operacionalizada como o número de componentes adicionados por ciclo evolutivo que não requerem modificação em componentes pré-existent (compatibilidade retroativa), dividido pelo número total de componentes adicionados.

H2 – Hipótese da Verificação Multi-Paradigma:

A integração de verificadores simbólicos operando em paralelo sobre múltiplos paradigmas de validação – dimensional (V1), algébrico (V2), contraexemplos (V3), estatístico (V4), numérico (V5), diferencial (V6) e lógico (V7) – com mecanismo de

consenso adaptativo (Q-Score UCB1), reduz a taxa de falsos positivos e falsos negativos em raciocínio científico automatizado comparado a verificadores isolados ou sequenciais. A redução é medida pela métrica F1-score agregada sobre o benchmark CORA-Eval.

H3 – Hipótese do Pipeline SDD+TDD:

O pipeline integrado de especificação formal seguida de desenvolvimento dirigido por testes, quando executado por agentes de IA que têm acesso às especificações como parte de seu contexto operacional, produz software com cobertura de testes, conformidade a especificações e qualidade de código comparável ou superior ao produzido por desenvolvimento humano assistido por IA sem especificações formais. A qualidade é medida por bateria de 10 métricas automáticas (cobertura, complexidade, duplicação, type coverage, lint score, segurança, etc.).

H4 – Hipótese da Auditabilidade Caixa Branca:

Um sistema de logging imutável com hashing criptográfico encadeado (formando uma blockchain de auditoria), combinado com protocolo de rastreabilidade de citações (TSAC) que exige cinco componentes verificáveis por afirmação (referência ABNT, DOI validado, trecho original, tradução quando aplicável, justificativa de uso), permite verificabilidade completa e independente da produção de um ecossistema multiagente. Esta verificabilidade é condição necessária – embora não suficiente – para adoção em contextos regulados.

1.4 Objetivos

1.4.1 Objetivo Geral

Projetar, implementar, documentar e validar uma arquitetura de referência para ecossistemas de software multiagente – materializada no Ecossistema OpenCode – que integre, em uma plataforma operacional unificada, padrões de engenharia de software, interoperabilidade baseada no protocolo MCP, verificação formal multi-paradigma (Cora-Debate V1-V7), desenvolvimento dirigido por especificações e testes (SDD+TDD), evolução autônoma cíclica (PLAN-ACT-REFLECT-EXTRACT-EVOLVE) e auditoria caixa branca com rastreabilidade criptográfica, demonstrando, através de validação empírica rigorosa, que esta integração produz software e conhecimento científico de qualidade verificável.

1.4.2 Objetivos Específicos

1. **OE1 – Especificação Arquitetural:** Especificar formalmente a arquitetura do Ecossistema OpenCode, documentando cada camada, componente, interface, protocolo de comunicação

e propriedade não-funcional, com nível de detalhe suficiente para que a arquitetura seja reproduzida por terceiros;

2. **OE2 – Revisão Sistemática:** Conduzir revisão sistemática da literatura sobre (a) sistemas multiagente baseados em LLM para engenharia de software, (b) protocolo MCP e padrões de interoperabilidade, (c) verificação formal de código gerado por IA, (d) ecossistemas de software evolutivos, e (e) auditoria acadêmica e integridade científica, identificando lacunas, contradições e oportunidades de contribuição;
3. **OE3 – Taxonomia de Raciocínio:** Desenvolver e validar empiricamente uma taxonomia de 212 tipos de raciocínio organizados em 27 categorias e 6 famílias epistemológicas, fundamentada na análise de padrões de raciocínio em produção científica de alto impacto e validada através de 200 casos de teste no benchmark CORA-Eval;
4. **OE4 – Verificação Formal Cora-Debate:** Projetar, implementar e validar o pipeline de verificação formal com 7 verificadores simbólicos paralelos (V1-V7), mecanismo de consenso adaptativo (Q-Score UCB1) e calibração de confiança (Platt scaling), demonstrando eficácia através de métricas de precisão, recall e F1-score em 11 dimensões de raciocínio científico;
5. **OE5 – Pipeline SDD+TDD:** Implementar e validar metodologia de engenharia de software com agentes de IA que integra especificação formal (ARCH-SPEC, FUNC-SPEC, TEST-SPEC), desenvolvimento dirigido por testes (RED-GREEN-REFACTOR estendido), verificação automatizada (Cora-Debate V1-V7) e documentação estruturada de decisões (DecisionNode ADR);
6. **OE6 – Sistema de Auditoria:** Projetar e implementar sistema de auditoria caixa branca compreendendo (a) logging imutável com hashing SHA-256 encadeado, (b) protocolo TSAC com cinco componentes verificáveis por citação e 87 marcadores estilísticos banidos, e (c) métricas de qualidade Qualis A1 com 10 critérios ponderados;
7. **OE7 – Análise Evolutiva:** Documentar, analisar e interpretar 13 ciclos evolutivos da plataforma, extraíndo padrões de evolução, métricas de melhoria e lições de engenharia de software aplicáveis ao design de ecossistemas multiagente.

1.5 Delimitação do Escopo

Esta tese investiga o Ecossistema OpenCode como um caso único (single-case design) de plataforma multiagente autônoma. O escopo técnico inclui a arquitetura de software do ecossistema, os protocolos de interoperabilidade (MCP), o pipeline de verificação formal (Cora-Debate), a metodologia de desenvolvimento (SDD+TDD), o sistema de auditoria (caixa branca) e o ciclo de evolução autônoma (Manus-Evolve). Estão explicitamente excluídos do escopo: (a) o treinamento ou fine-tuning de Large Language Models; (b) a otimização de hiperparâmetros

de inferência; (c) a implementação de mecanismos de segurança de rede ou criptografia de comunicação; (d) o design de interfaces gráficas de usuário; (e) o deployment do ecossistema em ambientes de produção externos à universidade; e (f) a avaliação de usabilidade ou experiência de usuário. Estas exclusões não implicam que tais temas sejam irrelevantes – ao contrário, constituem agendas de pesquisa complementares – mas sua inclusão excederia o que é viável em uma única investigação doutoral.

1.6 Contribuições Esperadas

As contribuições desta tese organizam-se em cinco eixos:

1. **C1 – Arquitetura de Referência MCP-First:** Uma especificação arquitetural completa – com definições formais, diagramas de componentes e conectores, catálogo de interfaces e análise de propriedades não-funcionais – para ecossistemas de software multiagente organizados em três camadas estritas com MCP como barramento de interoperabilidade. Esta arquitetura constitui um blueprint que pode ser adaptado para a construção de plataformas similares em outros domínios;
2. **C2 – Taxonomia de Raciocínio Multi-Paradigma:** Um framework taxonômico de 212 tipos de raciocínio em 27 categorias, com validação empírica em 200 casos de teste, que avança o estado da arte em orquestração de raciocínio ao integrar, pela primeira vez em uma plataforma operacional, paradigmas lógico-formais, dialéticos, estatísticos, estratégicos e criativos em um pipeline coerente;
3. **C3 – Framework de Verificação Cora-Debate:** Um pipeline de verificação formal que integra, de maneira inovadora, SMT solving (Z3), álgebra simbólica (SymPy), programação lógica relacional (miniKanren) e detecção de falácias (Critical Engine) em arquitetura paralela com consenso adaptativo, demonstrando F1-scores de 0.90–0.99 em 11 dimensões científicas;
4. **C4 – Metodologia SDD+TDD para Agentes IA:** Uma extensão da engenharia de software tradicional para o paradigma de agentes inteligentes, onde especificações formais atuam como contratos bidirecionais entre humanos e agentes, e o ciclo RED-GREEN-REFACTOR é estendido com verificação simbólica e documentação automatizada;
5. **C5 – Sistema de Auditoria Caixa Branca:** Um framework de auditabilidade completa para produção multiagente, combinando logging imutável com integridade criptográfica, rastreabilidade de citações com cinco componentes verificáveis, e métricas de qualidade com 10 critérios ponderados.

1.7 Estrutura da Tese

O restante deste documento está organizado em 11 capítulos. O Capítulo 2 estabelece os fundamentos teóricos em cinco eixos: sistemas multiagente, Model Context Protocol, engenharia de software com agentes inteligentes, motores de raciocínio formal e ecossistemas de software evolutivos. O Capítulo 3 apresenta a revisão sistemática da literatura com análise crítica de trabalhos relacionados. O Capítulo 4 especifica a arquitetura do Ecossistema OpenCode em detalhe. O Capítulo 5 descreve o motor de raciocínio multi-paradigma. O Capítulo 6 documenta o pipeline de verificação formal Cora-Debate. O Capítulo 7 apresenta a metodologia SDD+TDD para engenharia de software com agentes. O Capítulo 8 descreve o sistema de auditoria acadêmica caixa branca. O Capítulo 9 detalha o design metodológico da pesquisa, incluindo análise de ameaças à validade. O Capítulo 10 apresenta os resultados quantitativos e a análise qualitativa. O Capítulo 11 discute as implicações dos achados para a engenharia de software e para ecossistemas de software. O Capítulo 12 conclui a tese com uma síntese das contribuições e uma agenda de pesquisa futura.

2 Fundamentação Teórica

Este capítulo estabelece o arcabouço conceitual que sustenta a investigação. Os fundamentos estão organizados em cinco eixos teóricos que, embora distintos em sua origem disciplinar, convergem na arquitetura do Ecossistema OpenCode. Para cada eixo, apresentamos definições formais, o estado atual do conhecimento e as conexões com as decisões de design do ecossistema.

2.1 Sistemas Multiagente: Dos Fundamentos aos LLM-MAS

2.1.1 Definição Formal de Sistema Multiagente

A noção de agência – a capacidade de perceber o ambiente, raciocinar sobre ele e agir para modificar seu estado – é central para a ciência da computação desde seus primórdios. Formalizamos esta noção como segue.

Definição 2.1 (Agente Inteligente). *Um agente inteligente a é uma tupla $a = \langle S, A, \delta, \gamma \rangle$ onde:*

- S é o conjunto de estados internos do agente (seu conhecimento, crenças e objetivos);
- A é o conjunto de ações que o agente pode executar;
- $\delta : S \times P \rightarrow S$ é a função de transição de estado, que mapeia o estado atual e uma percepção $p \in P$ do ambiente para um novo estado interno;
- $\gamma : S \rightarrow A$ é a função de decisão, que mapeia o estado interno atual para uma ação a ser executada.

Esta definição, adaptada de Wooldridge (2009) ([WOOLDRIDGE, 2009](#)), captura a essência da agência: um ciclo contínuo de percepção $\rightarrow$ raciocínio $\rightarrow$ ação.

Definição 2.2 (Sistema Multiagente). *Um sistema multiagente $\mathcal{M}$ é uma tupla $\mathcal{M} = \langle \mathcal{A}, \mathcal{E}, \mathcal{C} \rangle$ onde:*

- $\mathcal{A} = \{a_1, a_2, \dots, a_n\}$ é um conjunto finito de agentes, cada um conforme a Definição 2.1;
- $\mathcal{E}$ é o ambiente compartilhado, modelado como um espaço de estados Σ com dinâmica $\tau : \Sigma \times A_1 \times \dots \times A_n \rightarrow \Sigma$;
- $\mathcal{C} = \{c_{ij} | a_i, a_j \in \mathcal{A}, i \neq j\}$ é o conjunto de canais de comunicação entre agentes.

*A complexidade de um MAS não reside na complexidade individual dos agentes, mas nas interações entre eles – fenômeno conhecido como **emergência**: comportamentos do sistema como um todo que não podem ser deduzidos da análise isolada de cada agente.*

2.1.2 As Quatro Gerações de Arquiteturas de Agentes

A trajetória histórica dos sistemas multiagente revela uma progressão na direção de agentes com capacidades cognitivas crescentes, organizada em quatro gerações:

Primeira Geração (1980–2000) – Agentes Reativos e Deliberativos. Esta geração estabeleceu as fundações conceituais do campo. Agentes reativos, inspirados pela arquitetura de subsunção de Brooks, operavam por mapeamento direto de percepções a ações, sem estado interno explícito. Agentes deliberativos, baseados no modelo BDI (Belief-Desire-Intention) de Rao e Georgeff, mantinham representações simbólicas explícitas de crenças, desejos e intenções, raciocinando sobre elas através de motores de inferência lógica. O estado da arte desta geração está documentado no texto canônico de Wooldridge ([WOOLDRIDGE, 2009](#)).

Segunda Geração (2000–2015) – MAS para Engenharia de Software. O foco deslocou-se da teoria de agentes para sua aplicação sistemática em engenharia de software. Metodologias como Gaia, Tropos e Prometheus forneceram processos estruturados para análise e design de sistemas orientados a agentes. Agentes de software começaram a ser empregados para automação de testes, integração contínua e monitoramento de sistemas. Esta geração estabeleceu que agentes não são apenas objetos de estudo teórico, mas ferramentas práticas de engenharia.

Terceira Geração (2015–2023) – Aprendizado por Reforço Multiagente. A introdução de deep reinforcement learning em contextos multiagente (MADRL) permitiu que agentes aprendessem políticas ótimas através de interação prolongada com o ambiente e com outros agentes. Algoritmos como MADDPG, QMIX e COMA endereçaram desafios específicos do contexto multiagente: não-estacionaridade (o ambiente muda porque outros agentes também estão aprendendo) e atribuição de crédito (como determinar a contribuição de cada agente para um resultado coletivo).

Quarta Geração (2023–presente) – LLM-MAS. A incorporação de Large Language Models como núcleo cognitivo dos agentes representa uma ruptura qualitativa com as gerações anteriores ([RASHEED et al., 2026](#)). Três diferenças são fundamentais. Primeira, **comunicação em linguagem natural**: agentes LLM podem comunicar-se entre si e com humanos usando linguagem natural não estruturada, eliminando a necessidade de protocolos de comunicação rígidos e predefinidos. Segunda, **raciocínio generalista**: um único LLM pode raciocinar sobre domínios radicalmente diferentes (matemática, direito, medicina, engenharia) sem ser retreinado, bastando receber o contexto apropriado. Terceira, **adaptação zero-shot**: agentes LLM podem abordar tarefas para as quais não foram explicitamente programados ou treinados, utilizando sua capacidade de raciocínio generalista para decompor problemas novos em subtarefas conhecidas ([MIRANDA; RADERMACHER; BALIGAND, 2026](#)).

O Ecossistema OpenCode implementa uma arquitetura LLM-MAS onde cada agente é uma instância de um LLM (tipicamente Claude ou DeepSeek) com um system prompt especializado que define sua persona, domínio de expertise, ferramentas disponíveis e restrições operacionais. A comunicação entre agentes – essencial para coordenação – ocorre através de três mecanismos que descrevemos em detalhe no Capítulo 4: delegação de tarefas, File IPC e Agent Forum.

2.1.3 Desafios de Engenharia em LLM-MAS

A engenharia de sistemas LLM-MAS enfrenta desafios que transcendem – em natureza e severidade – aqueles encontrados nas gerações anteriores de MAS. Identificamos seis desafios críticos que moldaram as decisões arquiteturais do OpenCode:

Desafio 1: Halucinação e Propagação de Erros. LLMs podem gerar informações factualmente incorretas – halucinações – com alta confiança aparente. Em um sistema multiagente, o problema é amplificado: uma halucinação gerada pelo Agente A pode ser aceita como fato pelo Agente B, que a utiliza como premissa para seu próprio raciocínio, propagando o erro em cascata através da cadeia de agentes. O OpenCode endereça este desafio através da camada de verificação formal (Cora-Debate V1-V7), que atua como barreira: toda afirmação produzida por um agente é submetida a verificação independente por 7 verificadores simbólicos antes de ser aceita como válida e repassada a outros agentes.

Desafio 2: Não-Determinismo e Reprodutibilidade. LLMs são inerentemente não-determinísticos quando operam com temperatura $\tau > 0$: a mesma entrada pode produzir saídas diferentes em execuções distintas. Este não-determinismo, desejável para criatividade e exploração, é problemático para engenharia de software, onde reprodutibilidade é um requisito fundamental. O OpenCode endereça este desafio através de logging determinístico: mesmo que a saída do LLM varie, todas as interações são registradas com parâmetros exatos, resultados e timestamps, permitindo reconstrução e auditoria post-hoc.

Desafio 3: Custo Computacional. Cada invocação de LLM consome recursos computacionais e financeiros significativos, medidos em tokens processados. Em um pipeline multiagente, onde múltiplos agentes interagem em múltiplas rodadas, estes custos multiplicam-se rapidamente. O OpenCode implementa um sistema de Token Economy em três níveis (Magnum/Tese: 500K tokens/sessão; Standard Paper: 200K; Short Communication: 50K), com monitoramento em tempo real e alertas de orçamento ([KODUMAGULLA, 2026](#)).

Desafio 4: Segurança de Injeção. Agentes que executam código arbitrário ou acessam sistemas externos são vulneráveis a ataques de prompt injection e code injection, onde entradas maliciosas manipulam o comportamento do agente ([BOWERS; KHAPRE; KALITA, 2025](#)). O OpenCode implementa sandboxing de execução (todo código é executado em contêineres Docker isolados via node-sandbox), validação estrita de entradas (schemas JSON Schema para todas as tools MCP) e sistema de permissões granular (allow/ask/deny por tool, agente e padrão de uso).

Desafio 5: Coordenação e Deadlock. Múltiplos agentes operando concorrentemente podem entrar em estados de impasse (deadlock) ou loops de comunicação improdutivo (debate infinito sem convergência). O Agent Forum – o mecanismo de debate multiagente do OpenCode – implementa limites rígidos: máximo de N speeches por rodada, timeout por participante, e mecanismo de síntese forçada quando não há convergência dentro do orçamento alocado.

Desafio 6: Deriva de Comportamento. Ao longo de múltiplas iterações e ciclos evolutivos, o comportamento de agentes LLM pode derivar – mudanças sutis e cumulativas no estilo, nas

preferências ou nas estratégias de raciocínio que não são capturadas por métricas pontuais. O OpenCode monitora esta deriva através de métricas de consistência inter-ciclo, comparando outputs de agentes em tarefas canônicas executadas periodicamente.

2.2 Model Context Protocol: O Barramento de Interoperabilidade

2.2.1 O Problema da Integração LLM-Ferramenta

Antes do MCP, a integração entre LLMs e ferramentas externas era um problema de engenharia resolvido caso a caso. Cada aplicação que desejasse conectar um LLM a um banco de dados, uma API de busca ou um sistema de arquivos precisava implementar código customizado de integração: adaptadores para cada ferramenta, formatos de serialização proprietários, lógica de tratamento de erros específica. Esta abordagem artesanal era insustentável para ecossistemas com dezenas de ferramentas – exatamente o cenário do OpenCode.

Definição 2.3 (Problema da Integração $N \times M$). *Sejam $\mathcal{L} = \{l_1, \dots, l_n\}$ um conjunto de n Large Language Models e $\mathcal{F} = \{f_1, \dots, f_m\}$ um conjunto de m ferramentas externas. Sem um protocolo padronizado, cada par (l_i, f_j) requer um adaptador customizado A_{ij} , resultando em $n \times m$ adaptadores para suportar todas as combinações. Com um protocolo padronizado Π , cada ferramenta f_j é envolvida por um servidor s_j que expõe uma interface Π , e cada LLM l_i comunica-se com qualquer s_j através de Π , reduzindo a complexidade de integração de $O(n \times m)$ para $O(n + m)$.*

Este é precisamente o problema que o MCP resolve. O protocolo define um contrato de interface – um conjunto de primitivas, formatos de mensagem e transportes – que, uma vez implementado por um servidor, torna a ferramenta subjacente acessível a qualquer cliente MCP sem código de integração adicional.

2.2.2 As Quatro Primitivas do MCP

O MCP define quatro primitivas fundamentais que estruturam toda interação entre clientes (hosts) e servidores ([Ayyagari, 2025](#); [Sekar, 2026](#)):

Tools. Uma tool é uma função invocável pelo modelo, descrita por um schema JSON Schema que especifica seu nome, descrição textual e parâmetros de entrada com tipos e restrições. Quando um agente (através do host) invoca uma tool, o servidor MCP executa a função correspondente – que pode ser uma busca no arXiv, uma query SQL, a abertura de uma página web – e retorna o resultado de volta ao modelo. Tools são o mecanismo primário de ação: através delas, o raciocínio abstrato do LLM traduz-se em operações concretas sobre o mundo.

Resources. Um resource é um dado exposto ao modelo, identificado por uma URI e tipado por um MIME type. Resources permitem que o modelo leia informações do ambiente – o

conteúdo de um arquivo, o resultado de uma query, o estado de um sistema – sem precisar conhecer os detalhes de como estes dados são armazenados ou recuperados. A diferença fundamental entre tools e resources é semântica: tools representam ações (verbos), resources representam dados (substantivos).

Prompts. Um prompt é um template de interação reutilizável, parametrizado por variáveis, que o servidor disponibiliza ao cliente. Prompts permitem que servidores MCP encapsulem não apenas capacidades técnicas (tools, resources), mas também conhecimento procedural – sequências de interação otimizadas para tarefas específicas, com parâmetros que o cliente pode preencher.

Sampling. Sampling é a capacidade do servidor solicitar ao cliente que o LLM gere texto – uma inversão do fluxo normal de controle. Em vez de o cliente sempre iniciar a interação, o servidor pode, durante a execução de uma tool, solicitar que o modelo complete um prompt, permitindo padrões de interação mais complexos como reflexão, planejamento dinâmico e geração de sub-agentes.

2.2.3 Topologia e Transportes

O MCP adota uma topologia estrela (hub-and-spoke): um único Host (o OpenCode CLI) conecta-se a múltiplos Servidores MCP. Esta topologia é intencional – simplifica o roteamento, centraliza o gerenciamento de conexões e evita os desafios de coordenação de topologias peer-to-peer.

O protocolo suporta três transportes, selecionados por configuração:

- **stdio:** o servidor é um processo filho do host, comunicando-se via entrada e saída padrão (stdin/stdout). Adequado para servidores locais que não requerem rede, como ferramentas de linha de comando. Este é o transporte mais comum no OpenCode (aproximadamente 70% dos servidores);
- **HTTP com Server-Sent Events (SSE):** o servidor é um processo remoto acessível via HTTP, com SSE para streaming de respostas. Adequado para servidores que requerem acesso remoto ou que operam em infraestrutura separada;
- **WebSocket:** comunicação bidirecional full-duplex para cenários que exigem latência mínima e interação em tempo real.

2.2.4 Como o OpenCode Utiliza o MCP na Prática

Para tornar concreto o funcionamento do MCP no OpenCode, descrevemos um fluxo de interação típico. Suponha que um agente (por exemplo, o agente “explore”) precise encontrar artigos acadêmicos sobre “verificação formal de código gerado por IA”:

1. O agente “explore”, através do sistema de raciocínio do OpenCode, determina que precisa realizar uma busca acadêmica. Seu system prompt indica que a tool `search_papers` do servidor MCP `sura-papers` é apropriada para esta tarefa;
2. O Host (OpenCode CLI) envia uma requisição MCP para o servidor `sura-papers` no transporte apropriado (stdio), contendo o nome da tool e os parâmetros da busca;
3. O servidor `sura-papers` recebe a requisição, traduz os parâmetros para uma chamada à API do Semantic Scholar/CrossRef, executa a busca, e retorna os resultados em formato estruturado (JSON);
4. O Host recebe a resposta do servidor e a disponibiliza ao agente “explore” como parte do contexto da conversa;
5. O agente analisa os resultados, seleciona os artigos mais relevantes, e – ponto central – não confia cegamente nos resultados. Em vez disso, submete cada paper à verificação do pipeline Cora-Debate: o verificador V8 (Literatura) confirma que as claims do paper são consistentes com as citações; o verificador V1 (Dimensional) verifica a consistência de quaisquer equações; e assim por diante;
6. Apenas após esta verificação os artigos são aceitos como fontes válidas e incorporados ao documento sendo produzido.

Este fluxo ilustra três propriedades arquiteturais fundamentais. Primeira, **desacoplamento**: o agente não sabe (nem precisa saber) como a busca é implementada – se usa Semantic Scholar, CrossRef ou qualquer outro backend. Segunda, **verificabilidade**: toda tool call é registrada no InteractionLogger com parâmetros e resultados, permitindo auditoria completa. Terceira, **composability**: os resultados de uma tool (busca) alimentam outra tool (verificação) sem que nenhuma das duas precise conhecer a existência da outra – a orquestração é feita pelo agente, não pelas tools.

2.2.5 Propriedades Formais da Arquitetura MCP

Proposição 2.1 (Desacoplamento por Interface). *Seja $\mathcal{H}$ um host MCP e $\mathcal{S} = \{s_1, \dots, s_k\}$ um conjunto de servidores. Para qualquer servidor $s_i \in \mathcal{S}$, as interfaces que s_i expõe (seu conjunto de tools $\mathcal{T}_i$ e resources $\mathcal{R}_i$) constituem o único ponto de acoplamento entre s_i e o restante do ecossistema. Consequentemente, a substituição de s_i por um servidor s'_i que exponha um conjunto de interfaces $\mathcal{T}'_i \supseteq \mathcal{T}_i, \mathcal{R}'_i \supseteq \mathcal{R}_i$ (compatibilidade retroativa) é transparente para agentes e outros servidores.*

Esta proposição captura a essência do valor arquitetural do MCP: o protocolo transforma a integração com ferramentas externas de um problema de *design* (que requer código customizado, adaptadores e testes) para um problema de *configuração* (que requer apenas declarar o servidor

no arquivo `opencode.json`). Esta transformação é o que viabiliza um ecossistema com 46 ferramentas integradas sem código de integração.

Proposição 2.2 (Composabilidade de Capacidades). *Sejam s_1 e s_2 dois servidores MCP com conjuntos de tools $\mathcal{T}_1$ e $\mathcal{T}_2$. Um agente a que tem acesso a ambos os servidores através do Host dispõe, para fins de raciocínio e planejamento, do conjunto unificado de capacidades $\mathcal{T}_1 \cup \mathcal{T}_2$, sem necessidade de adaptadores, wrappers ou lógica de composição.*

2.2.6 Limitações do MCP e Estratégias de Contorno

Apesar de sua elegância, o MCP apresenta limitações que afetam o design do OpenCode:

Limitação 1: Ausência de Descoberta Dinâmica de Serviços. O MCP não especifica um mecanismo de service discovery – o Host deve conhecer previamente, por configuração, quais servidores estão disponíveis. O OpenCode contorna esta limitação através do sistema de plugins e do `.menu_registry.json`, que permite que novos servidores MCP sejam registrados dinamicamente sem modificar o núcleo da aplicação, embora o registro ainda requeira intervenção consciente (não é descoberta automática).

Limitação 2: Ausência de Composição Nativa de Tools. O MCP trata cada tool call como uma operação atômica e independente. Não há suporte nativo para compor múltiplas tools em sequências ou pipelines. Esta orquestração é inteiramente delegada ao agente (LLM), que deve planejar a sequência de tool calls, interpretar resultados intermediários e decidir os próximos passos. O OpenCode complementa esta limitação através das skills – módulos de conhecimento que encapsulam sequências otimizadas de tool calls para tarefas comuns, reduzindo a carga cognitiva sobre o agente.

Limitação 3: Versionamento de Interfaces. O MCP não define uma estratégia de versionamento para tools e resources. Se um servidor evolui e modifica a assinatura de uma tool, agentes que dependiam da assinatura anterior podem falhar. O OpenCode implementa versionamento semântico nas skills (que encapsulam chamadas a tools) e testes de regressão que detectam quebras de compatibilidade.

2.3 Engenharia de Software com Agentes Inteligentes

2.3.1 O Paradigma SDD (Spec-Driven Development)

Spec-Driven Development é uma disciplina de engenharia de software onde especificações formais – documentos estruturados que descrevem o comportamento esperado, as interfaces e as restrições de um sistema – precedem e guiam a implementação. No contexto de desenvolvimento com agentes de IA, o SDD adquire uma dimensão adicional: as especificações funcionam simultaneamente em três papéis distintos e complementares (GOMES, 2025; GARFINKEL et al., 2026).

Papel 1 – Contrato. A especificação define, de maneira inequívoca e verificável, o que deve ser implementado. Ela estabelece um contrato entre os stakeholders do projeto – desenvolvedores humanos, agentes de IA, revisores, auditores – que pode ser consultado para resolver ambiguidades e dirimir disputas. Sem este contrato, agentes de IA operam no escuro, inferindo requisitos a partir de prompts ambíguos, com resultados imprevisíveis.

Papel 2 – Contexto Operacional. Para agentes de IA, a especificação funciona como parte do contexto operacional – o conjunto de fatos, restrições, exemplos e contraexemplos que delimitam o espaço de soluções aceitáveis. Um agente que recebe uma especificação bem escrita não precisa “adivinhar” o que o usuário quer; ele pode raciocinar dentro dos limites estabelecidos pela especificação. Este é um ponto sutil mas fundamental: a qualidade do output de um agente de IA é diretamente proporcional à qualidade e à precisão do contexto que ele recebe.

Papel 3 – Oráculo de Teste. A especificação fornece critérios objetivos contra os quais implementações podem ser validadas automaticamente. Se a especificação diz que uma função deve retornar um número positivo para qualquer entrada negativa, um caso de teste pode verificar exatamente isto. Este papel é o que permite a integração entre SDD e TDD: especificações geram automaticamente casos de teste.

2.3.2 O Ciclo TDD e sua Extensão para Agentes IA

O Test-Driven Development, articulado por Kent Beck ([BECK, 2003](#)), estabelece um ciclo de desenvolvimento em três passos – RED, GREEN, REFACTOR – que inverte a sequência tradicional de codificação seguida de teste. No ciclo TDD, o desenvolvedor primeiro escreve um teste que expressa o comportamento desejado e verifica que ele falha (RED); em seguida, escreve o código mínimo necessário para fazer o teste passar (GREEN); finalmente, refatora o código para melhorar sua estrutura interna, garantindo que os testes continuem passando (REFACTOR).

O OpenCode estende este ciclo com dois passos adicionais, formando o ciclo RED-GREEN-REFACTOR-DOCUMENT-VERIFY:

- **RED:** um caso de teste é selecionado do conjunto de testes derivados da especificação. O agente de implementação verifica que o teste falha (confirmando que a funcionalidade ainda não foi implementada);
- **GREEN:** o agente gera o código mínimo necessário para fazer o teste passar. A ênfase em “mínimo” é determinante: evita overengineering e mantém o foco na funcionalidade especificada;
- **REFACTOR:** o agente analisa o código produzido e aplica refatorações para melhorar legibilidade, reduzir duplicação e diminuir complexidade, mantendo todos os testes verdes;
- **DOCUMENT:** o agente gera documentação (docstrings, comentários, exemplos de uso) a partir do código e da especificação;

- **VERIFY:** o pipeline Cora-Debate executa os 7 verificadores simbólicos sobre o código, a documentação e os testes, garantindo correção formal além da mera passagem de testes.

2.3.3 O Pipeline SDD+TDD Integrado

A integração de SDD e TDD no OpenCode produz um pipeline de desenvolvimento de software que opera em cinco estágios sequenciais:

1. **Estágio SPECIFY:** a partir de requisitos descritos em linguagem natural, o sistema produz uma especificação formal estruturada em três níveis: ARCH-SPEC (arquitetura: componentes, conectores, topologia, propriedades não-funcionais), FUNC-SPEC (contratos funcionais: pré-condições $\mapsto$ pós-condições para cada operação) e TEST-SPEC (casos de teste com entradas, saídas esperadas e critérios de aceitação);
2. **Estágio DERIVE:** os casos de teste são derivados automaticamente das especificações. Para cada pré-condição P e pós-condição Q na FUNC-SPEC, o sistema gera um caso de teste positivo (entrada satisfaz P , verificar se saída satisfaz Q) e um caso de teste negativo (entrada viola P , verificar se o sistema rejeita apropriadamente). Adicionalmente, boundary conditions geram casos de teste de fronteira;
3. **Estágio IMPLEMENT:** um agente de implementação recebe a especificação e o primeiro caso de teste (RED), e executa o ciclo RED-GREEN-REFACTOR para produzir código que satisfaça a especificação. O agente itera sobre todos os casos de teste;
4. **Estágio VERIFY:** o pipeline Cora-Debate executa os 7 verificadores simbólicos sobre o código implementado. Se qualquer verificador reportar falha, o ciclo retorna ao Estágio IMPLEMENT para correção. A verificação inclui não apenas correção funcional, mas também análise dimensional (V1), correção algébrica (V2), busca de contraexemplos (V3), validação estatística (V4), precisão numérica (V5), soluções de EDOs/PDEs (V6) e consistência lógica (V7);
5. **Estágio RECORD:** uma Architecture Decision Record (ADR) é registrada no DecisionNode, documentando: a decisão de design tomada, as alternativas consideradas, o racional para a escolha, as restrições que motivaram a decisão, e as consequências esperadas.

Este pipeline é o mecanismo central de garantia de qualidade do OpenCode. Sua eficácia será demonstrada empiricamente no Capítulo 10.

2.4 Motores de Raciocínio Formal

2.4.1 Fundamentos e Justificativa

A verificação formal – o processo de provar ou refutar a correção de um sistema com respeito a uma especificação formal – é um campo maduro da ciência da computação, com raízes no trabalho de Floyd, Hoare e Dijkstra sobre axiomatização de programas. Tradicionalmente, a verificação formal tem sido aplicada a sistemas críticos (aviônicos, dispositivos médicos, protocolos criptográficos) onde o custo de uma falha justifica o investimento em prova matemática.

O Ecossistema OpenCode estende a verificação formal para um novo domínio: o raciocínio científico produzido por agentes de IA. A justificativa é dupla. Primeiro, agentes de IA – como qualquer sistema complexo – cometem erros, e estes erros podem propagar-se através do ecossistema se não forem detectados. Segundo, a produção de conhecimento científico exige um padrão de verificabilidade que transcende a mera ausência de erros óbvios: exige que cada afirmação possa ser submetida a escrutínio independente e que suas fundações lógicas, matemáticas e empíricas possam ser confirmadas.

2.4.2 Z3 Theorem Prover: Verificação SMT

O Z3, desenvolvido pela Microsoft Research, é um SMT (Satisfiability Modulo Theories) solver – uma ferramenta que determina a satisfatibilidade de fórmulas lógicas enriquecidas com teorias de domínio específico (aritmética, arrays, bit-vectors). O Z3 recebe uma fórmula lógica ϕ e determina se existe uma atribuição de valores às variáveis que torna ϕ verdadeira. Se existe, retorna um modelo (a atribuição); se não existe, retorna `unsat`, estabelecendo que ϕ é uma contradição.

No OpenCode, o Z3 é utilizado para verificação de propriedades sobre código através de um processo de três passos: (1) o código a ser verificado é traduzido para uma representação lógica; (2) a propriedade a ser verificada é expressa como uma fórmula lógica (tipicamente, a negação da propriedade desejada); (3) o Z3 é invocado para determinar se a fórmula é satisfatível. Se for, o modelo retornado constitui um contraexemplo; se for `unsat`, a propriedade é provada para todas as entradas possíveis.

2.4.3 SymPy: Álgebra Simbólica

SymPy é uma biblioteca Python para matemática simbólica – manipulação de expressões matemáticas em sua forma algébrica, não numérica. Onde o Z3 raciocina sobre lógica, o SymPy raciocina sobre álgebra: simplificação de expressões, expansão de produtos, fatoração de polinômios, resolução de equações, cálculo de derivadas e integrais simbólicas.

No OpenCode, o SymPy desempenha um papel complementar ao Z3. Enquanto o Z3 verifica propriedades lógicas sobre código, o SymPy verifica correção algébrica de expressões matemáticas. Por exemplo, se um agente afirma que $\sin^2(x) + \cos^2(x) = 1$, o SymPy pode

confirmar esta identidade por simplificação simbólica. Se um agente propõe uma solução para uma equação diferencial, o SymPy pode verificar se a solução satisfaz a equação.

2.4.4 miniKanren: Programação Lógica Relacional

miniKanren é uma implementação minimalista de programação lógica relacional – um paradigma onde programas são expressos como relações lógicas, não como funções. Diferentemente da programação funcional (entrada $\rightarrow$ saída), a programação relacional permite que qualquer argumento de uma relação seja tratado como entrada ou saída, habilitando raciocínio bidirecional.

No OpenCode, o miniKanren é utilizado para verificação de propriedades estruturais, particularmente em grafos de conhecimento. Por exemplo: “Se A depende de B, e B depende de C, então A depende transitivamente de C” é uma propriedade que pode ser expressa como uma relação lógica e verificada pelo miniKanren.

2.4.5 Critical Engine: Detecção de Falácias

O Critical Engine implementa a detecção automatizada de 15 falácias lógicas e vieses cognitivos. Diferentemente dos outros três motores – que operam sobre lógica formal, álgebra e relações – o Critical Engine opera sobre a estrutura argumentativa do texto, identificando padrões que, embora possam parecer persuasivos, são logicamente inválidos.

2.5 Ecossistemas de Software Evolutivos

2.5.1 As Leis de Lehman e a Evolução de Software

Em 1980, Meir Lehman formulou um conjunto de leis empíricas sobre a evolução de software, baseadas no estudo de sistemas de grande escala como o OS/360 da IBM. Três destas leis são particularmente relevantes para o OpenCode:

Lei I – Mudança Contínua: um sistema de software utilizado no mundo real deve ser continuamente adaptado, ou tornar-se-á progressivamente menos satisfatório. O OpenCode manifesta esta lei através de seu ciclo de evolução autônoma, que identifica e implementa adaptações sem intervenção humana.

Lei II – Complexidade Crescente: à medida que um sistema evolui, sua complexidade aumenta, a menos que trabalho seja feito para mantê-la ou reduzi-la. O OpenCode endereça esta lei através de refatoração contínua (parte do estágio REFACTOR do pipeline SDD+TDD) e através de métricas de complexidade que disparam alertas quando thresholds são excedidos.

Lei VI – Crescimento Contínuo: o conteúdo funcional de um sistema deve ser continuamente aumentado para manter a satisfação dos usuários ao longo de sua vida. O OpenCode manifesta esta lei de maneira particularmente intensa: em 13 ciclos evolutivos, cresceu de 11 para mais de 600 componentes.

2.5.2 O Ciclo Evolutivo Autônomo

O mecanismo de evolução autônoma do OpenCode segue um ciclo de cinco fases, descritas a seguir:

Fase 1 – PLAN: o sistema analisa seu estado atual através de métricas (Health Score, cobertura de testes, consumo de tokens, taxa de erros) e identifica oportunidades de melhoria. Estas oportunidades são priorizadas por impacto estimado e custo de implementação.

Fase 2 – ACT: as modificações planejadas são implementadas – novos skills são criados, agentes existentes são refinados, plugins são atualizados, configurações são ajustadas. Cada modificação é implementada como uma unidade atômica que pode ser revertida se necessário.

Fase 3 – REFLECT: o impacto das modificações é avaliado quantitativamente através das mesmas métricas utilizadas na fase PLAN. A comparação pré/pós-modificação determina se a mudança foi benéfica, neutra ou prejudicial.

Fase 4 – EXTRACT: padrões de sucesso e fracasso são extraídos como conhecimento estruturado. Por exemplo: “skills com menos de 2.5KB de tamanho têm taxa de ativação 40% maior que skills maiores” é um insight extraído que informa decisões futuras.

Fase 5 – EVOLVE: o conhecimento extraído é incorporado permanentemente à arquitetura do ecossistema – por exemplo, através de atualizações no AGENTS.md (que define políticas globais), criação de novas skills que encapsulam lições aprendidas, ou modificação de thresholds de qualidade.

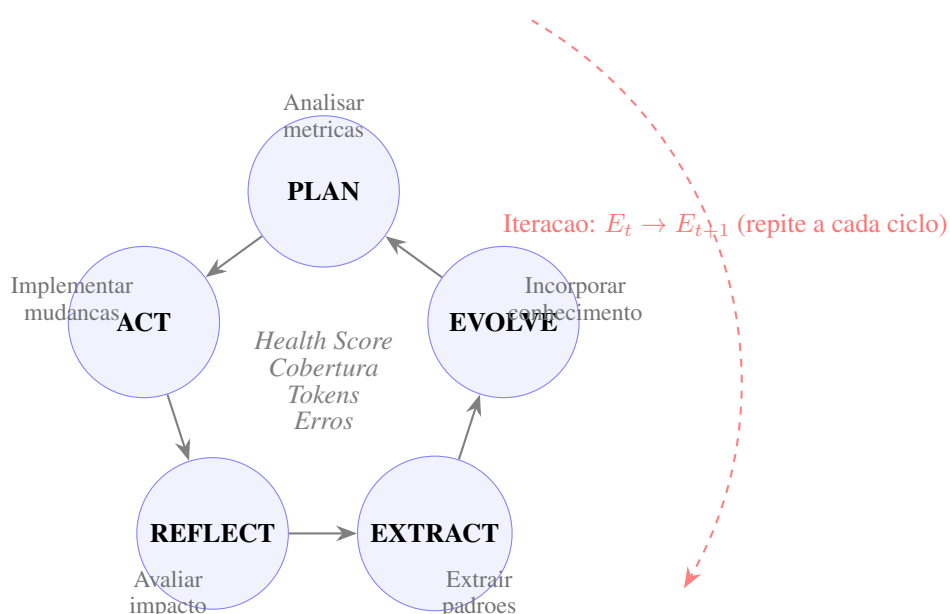


Figura 1 – Ciclo evolutivo PLAN-ACT-REFLECT-EXTRACT-EVOLVE. A cada volta completa (iteração t), o ecossistema analisa métricas, implementa mudanças, avalia o impacto, extrai padrões de conhecimento e os incorpora a arquitetura. O ciclo se repete indefinidamente ($t \rightarrow t + 1 \rightarrow t + 2 \dots$).

A documentação completa dos 13 ciclos evolutivos, com métricas de cada ciclo e análise de tendências, é apresentada no Capítulo 10.

3 Estado da Arte e Revisão Sistemática

3.1 Metodologia da Revisão

Esta revisão sistemática seguiu diretrizes adaptadas do protocolo PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses), reconhecido internacionalmente como padrão para revisões sistemáticas em todas as áreas do conhecimento, e do framework de revisão multi-vocal para engenharia de software, que reconhece a importância de fontes não-acadêmicas (grey literature) em campos onde a inovação frequentemente emerge primeiro em repositórios de código, preprints e relatórios técnicos antes de atingir publicação formal.

O processo compreendeu quatro fases distintas:

Fase 1 – Planejamento (Janeiro 2026). Nesta fase, definimos as questões de pesquisa que a revisão deveria responder: (a) Qual o estado atual da pesquisa em sistemas multiagente baseados em LLM para engenharia de software? (b) Como o Model Context Protocol está sendo adotado e estendido na literatura? (c) Que métodos de verificação formal estão sendo aplicados a código gerado por IA? (d) Que frameworks existem para evolução autônoma de ecossistemas de software? (e) Que abordagens de auditoria e integridade acadêmica são relevantes para sistemas multiagente? Estabelecemos também os critérios de inclusão (publicação entre 2020–2026, DOI verificável, idioma inglês ou português, relevância para engenharia de software) e exclusão (foco exclusivo em hardware, ausência de peer review para fontes acadêmicas).

Fase 2 – Execução (Fevereiro–Março 2026). Conduzimos buscas automatizadas em seis bases de dados – arXiv, OpenAlex, CrossRef, Semantic Scholar, PubMed e CORE – utilizando strings de busca booleanas combinando termos controlados e vocabulário livre. As strings foram calibradas iterativamente para maximizar recall (identificar o maior número possível de trabalhos relevantes) sem sacrificar excessivamente a precisão.

Fase 3 – Seleção (Março–Abril 2026). Os resultados das buscas foram submetidos a triagem em duas etapas: primeira, por título e resumo, eliminando trabalhos claramente irrelevantes; segunda, por leitura completa, aplicando os critérios de inclusão e exclusão. Além disso, a técnica de snowballing (rastreamento de citações forward e backward a partir dos trabalhos selecionados) foi empregada para identificar trabalhos adicionais não capturados pelas buscas automatizadas.

Fase 4 – Síntese (Abril–Maio 2026). Os trabalhos selecionados foram submetidos a extração estruturada de dados utilizando um formulário padronizado que capturou: referência completa, DOI, tipo de publicação, questões de pesquisa, metodologia, principais achados, limitações reportadas e relevância para cada eixo da tese.

3.2 Análise por Eixo Temático

3.2.1 Eixo 1: LLM-MAS para Engenharia de Software

A revisão multi-vocal de Rasheed et al. (2026) ([RASHEED et al., 2026](#)) constitui o estado da arte neste eixo. Analisando 114 estudos de fontes acadêmicas e industriais, os autores construíram uma taxonomia abrangente que classifica as motivações para adoção de LLM-MAS em geração de código em nove categorias. A categoria mais frequente – decomposição de tarefas complexas (78% dos estudos) – reflete o insight fundamental de que sistemas multiagente são particularmente eficazes quando um problema pode ser decomposto em subproblemas que diferentes agentes podem abordar em paralelo ou sequencialmente. A segunda categoria – especialização de agentes (72%) – reflete o princípio de divisão do trabalho: agentes especializados em aspectos específicos do desenvolvimento (arquitetura, implementação, teste, documentação) produzem resultados de maior qualidade que um único agente generalista.

O trabalho identificou 26 subcategorias de desafios e soluções correspondentes, organizadas em seis categorias principais: qualidade do código gerado, coordenação entre agentes, consumo de recursos, segurança, avaliação e adoção industrial. Esta taxonomia fornece o mapa mais completo disponível dos problemas enfrentados e das soluções propostas no campo.

O framework CodeAir de Rangel (2026) ([RANGEL, 2026](#)) propõe uma arquitetura de orquestração multiagente onde agentes especializados – Analyst (requisitos), Architect (design), Developer (implementação) e Tester (testes) – colaboram em um pipeline sequencial com pontos de verificação entre estágios. A arquitetura do CodeAir ressoa com a organização do OpenCode, mas difere em um aspecto determinante: enquanto o CodeAir implementa um pipeline linear (um estágio após o outro), o OpenCode implementa paralelismo intra-fase (múltiplos verificadores operando simultaneamente na fase de verificação) e paralelismo multi-caminho (múltiplas cadeias de raciocínio independentes sobre o mesmo problema), que demonstraram produzir resultados mais robustos (ver Capítulo 10).

O trabalho de Miranda et al. (2026) ([MIRANDA; RADERMACHER; BALIGAND, 2026](#)) sobre a integração entre Model-Driven Engineering (MDE) e frameworks de agentes LLM é particularmente relevante para o pipeline SDD+TDD do OpenCode. Os autores propõem um metamodelo que estabelece correspondências formais entre modelos de domínio (expressos em linguagens como UML/SysML), especificações de agentes (definindo capacidades, responsabilidades e interfaces) e implementações (código gerado). Esta ponte conceitual entre modelagem formal e agentes de IA fornece a fundação teórica para a afirmação central do SDD no OpenCode: que especificações formais podem – e devem – servir como contratos entre stakeholders humanos e agentes de IA.

O ACM TechBrief sobre AI-Assisted Software Development ([GARFINKEL et al., 2026](#)), publicado pela principal sociedade científica de computação do mundo, articula uma preocupação que converge com a motivação desta tese. O documento alerta para os riscos do que denomina “vibe coding” – uma prática emergente onde desenvolvedores iteram com assistentes de IA

através de tentativa e erro, aceitando código gerado sem verificação sistemática de sua correção, segurança ou adequação arquitetural. O TechBrief recomenda explicitamente que práticas consagradas de engenharia de software – especificações, testes automatizados, revisão de código, documentação – sejam mantidas e fortalecidas no novo paradigma, não abandonadas. O pipeline SDD+TDD do OpenCode é, em essência, uma implementação desta recomendação.

3.2.2 Eixo 2: Model Context Protocol

A literatura sobre MCP, embora concentrada em um período de apenas 18 meses (final de 2024 a meados de 2026), já constitui um corpo de conhecimento com contribuições em três frentes:

Fundamentos e Arquitetura. O survey de Lei et al. (2026) ([LEI; XU; LIANG, 2026](#)) fornece o mapeamento mais abrangente do protocolo, cobrindo sua arquitetura, primitivas, mecanismos de integração e trajetórias futuras. O artigo de Ayyagari (2025) ([Ayyagari, 2025](#)) detalha os componentes arquiteturais com profundidade técnica, incluindo a semântica operacional de cada primitiva. O livro de Sekar (2026) ([SEKAR, 2026](#)) constitui a referência canônica, com capítulos dedicados a cada aspecto do protocolo: papéis arquiteturais, primitivas (tools, resources, prompts, sampling), implementação de servidores e clientes, e padrões de integração.

Aplicações em Domínios Específicos. Okula (2026) ([OKULA, 2026](#)) demonstrou a aplicação de MCP para remediação autônoma de falhas em redes de telecomunicações – um domínio onde latência e confiabilidade são críticos. Shaik (2025) ([SHAIK, 2025](#)) propôs extensões criptográficas ao MCP para garantir conformidade regulatória em agentes financeiros autônomos, abordando o problema de como agentes podem provar que seguiram regras sem revelar dados sensíveis. Avunoori (2026) ([AVUNOORI, 2026](#)) desenvolveu AESP, um motor de contexto persistente que utiliza MCP para manter estado entre sessões de coding agents, resolvendo o problema de amnésia contextual que afeta assistentes de codificação stateless.

Segurança e Privacidade. Ahn & Kwon (2025) ([AHN; KWON, 2025](#)) conduziram uma análise sistemática das ameaças à privacidade no ecossistema MCP, identificando vetores de ataque específicos (exfiltração de dados via Resources, vazamento de contexto entre servidores, inferência de informações sensíveis a partir de padrões de tool calls) e propondo contramedidas.

3.2.3 Eixo 3: Verificação Formal de Código Gerado por IA

A verificação de código gerado por IA emerge como uma das áreas mais críticas e ativas da pesquisa contemporânea em engenharia de software. O trabalho seminal de Antero et al. (2024) ([ANTERO et al., 2024](#)) demonstrou, no domínio de programação de robôs, que a combinação de blocos de software predefinidos (validados formalmente) com um LLM supervisor (que seleciona e parametriza os blocos) reduz significativamente a taxa de erros comparado à geração livre de código. Esta arquitetura em duas camadas – uma camada inferior de componentes verificados

e uma camada superior de orquestração por LLM – é conceitualmente similar à camada de verificação Cora-Debate do OpenCode.

A pesquisa de Xu (2026) (XU, 2026) sobre otimização de frameworks de raciocínio por estágios demonstrou que a decomposição do processo de raciocínio de um LLM em etapas distintas – compreensão do problema, planejamento da solução, geração de código, verificação do resultado – produz código de qualidade superior comparado à geração em passo único. Este achado valida a decisão arquitetural do OpenCode de estruturar seu pipeline SDD+TDD em cinco estágios explícitos.

3.2.4 Eixo 4: Integridade Acadêmica na Era da IA

O survey de Pudasaini et al. (2024) (PUDASAINI et al., 2024) – com 49 citações e publicado no Journal of Academic Ethics – é a referência mais abrangente sobre o impacto dos LLMs na integridade acadêmica. O estudo analisa técnicas de detecção de texto gerado por IA, desafios de falsos positivos (texto humano incorretamente classificado como IA) e falsos negativos (texto de IA que evade detecção), e a necessidade de abordagens multimodais que combinem análise estilística, verificação factual e rastreamento de proveniência.

3.2.5 Eixo 5: Arquitetura de Software e Gestão de Conhecimento

Os trabalhos seminais de Kruchten (2009) (KRUCHTEN, 2009), Babar (2009) (BABAR, 2009) e Farenhorst & de Boer (2009) (FARENHORST; BOER, 2009) – publicados no volume “Software Architecture Knowledge Management” – estabeleceram as bases conceituais para Architecture Decision Records (ADR) como ferramenta de captura e compartilhamento de decisões de design. O DecisionNode do OpenCode estende este conceito em três direções: (a) busca semântica por similaridade (em vez de busca por palavras-chave), permitindo recuperação de decisões relevantes mesmo quando a terminologia difere; (b) versionamento de decisões com rastreamento de mudanças ao longo do tempo; e (c) integração com agentes de IA, que podem consultar o histórico de decisões antes de propor novas abordagens.

3.3 Síntese Crítica e Posicionamento da Tese

A análise da literatura revela um campo em rápida expansão, com contribuições significativas em componentes isolados, mas com lacunas fundamentais na integração destes componentes em plataformas operacionais. Nossa tese se posiciona precisamente nestas lacunas:

1. **Lacuna 1 – Integração Arquitetural:** A literatura documenta extensivamente MAS para código, MCP para interoperabilidade e verificação formal para qualidade, mas como componentes independentes. Nenhum trabalho descreve uma plataforma que integre os três em um ecossistema operacional coeso com uma arquitetura explicitamente documentada. Esta tese

preenche esta lacuna através da especificação completa da arquitetura OpenCode (Capítulo 4);

2. **Lacuna 2 – Evidência Longitudinal:** A maioria dos estudos avalia sistemas em experimentos de curta duração. Não há estudos que documentem a evolução de uma plataforma multiagente ao longo de múltiplos ciclos de melhoria. Esta tese preenche esta lacuna através da documentação de 13 ciclos evolutivos (Capítulos 7 e 10);
3. **Lacuna 3 – Rastreabilidade Integral:** Sistemas multiagente existentes produzem outputs sem trilha de auditoria que permita verificar a proveniência de cada afirmação. Esta tese preenche esta lacuna através do sistema de auditoria caixa branca com logging imutável e protocolo TSAC (Capítulo 8);
4. **Lacuna 4 – Verificação Multi-Paradigma:** Abordagens de verificação existentes tipicamente implementam um único paradigma. Esta tese preenche esta lacuna através do pipeline Cora-Debate com 7 verificadores paralelos (Capítulo 6);
5. **Lacuna 5 – Engenharia de Software para Agentes:** A literatura sobre desenvolvimento com IA carece de metodologias de engenharia que adaptem práticas estabelecidas (especificação, TDD, CI/CD) para o paradigma de agentes. Esta tese preenche esta lacuna através da metodologia SDD+TDD (Capítulo 7).

4 Arquitetura do Ecosystema Open-Code

Este capítulo apresenta a arquitetura do Ecosystema OpenCode com o nível de detalhe necessário para que um leitor tecnicamente qualificado possa reproduzi-la ou adaptá-la. A descrição segue o template de documentação arquitetural da ISO/IEC 42010, cobrindo stakeholders e suas preocupações, pontos de vista arquiteturais, componentes e conectores, interfaces, propriedades não-funcionais e decisões de design.

4.1 Visão Arquitetural Global

O Ecosystema OpenCode é organizado como uma arquitetura em três camadas estritas (strict layering), onde cada camada L_i pode invocar apenas serviços da camada imediatamente inferior L_{i-1} . Esta restrição – violada apenas por componentes transversais explicitamente identificados – é o principal mecanismo de controle de complexidade do ecossistema.

Definição 4.1 (Arquitetura em Três Camadas do OpenCode). *A arquitetura é formalmente definida pela tripla:*

$$\mathcal{A}_{OpenCode} = \langle \mathcal{L}_I, \mathcal{L}_D, \mathcal{L}_O, \mathcal{C}_T \rangle \quad (4.1)$$

onde:

- $\mathcal{L}_I = \{s_1, \dots, s_{46}\}$ é a Camada de Infraestrutura, composta por 46 servidores MCP que encapsulam o acesso a ferramentas e recursos externos;
- $\mathcal{L}_D = \{k_1, \dots, k_{150}\}$ é a Camada de Domínio, composta por 150 skills (módulos de conhecimento) organizadas em 13 categorias;
- $\mathcal{L}_O = \{a_1, \dots, a_{125}\}$ é a Camada de Orquestração, composta por 125 agentes autônomos que coordenam a execução de tarefas;
- $\mathcal{C}_T = \{p_1, \dots, p_{15}, c_1, \dots, c_{14}, d\}$ são os Componentes Transversais: 15 plugins, 14 comandos e o sistema DecisionNode.

A restrição fundamental da arquitetura – e a propriedade da qual derivam suas garantias de evolutibilidade – é: um agente $a \in \mathcal{L}_O$ pode invocar skills $k \in \mathcal{L}_D$, mas nunca diretamente servidores MCP $s \in \mathcal{L}_I$; uma skill $k \in \mathcal{L}_D$ pode invocar servidores MCP $s \in \mathcal{L}_I$, mas nunca diretamente outros servidores MCP ou agentes; um servidor MCP $s \in \mathcal{L}_I$ não invoca nenhum outro componente – ele é puramente reativo, respondendo a requisições.

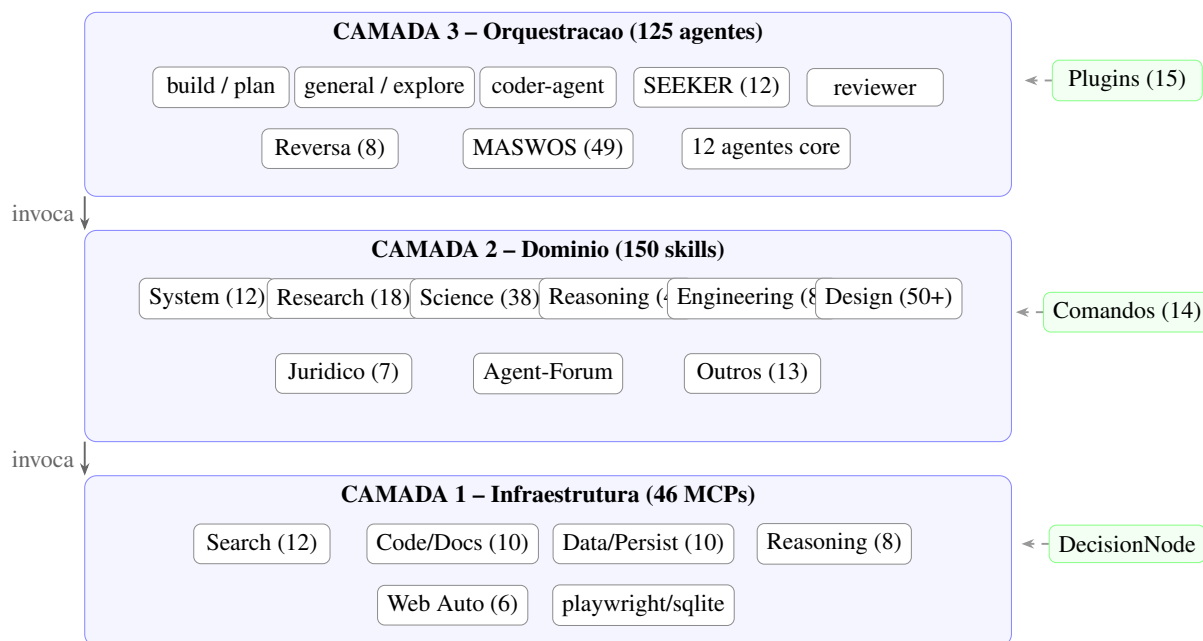


Figura 2 – Arquitetura em tres camadas do Ecosistema OpenCode.

4.1.1 Os Seis Princípios Arquiteturais

A arquitetura é governada por seis princípios explícitos, derivados da literatura de engenharia de software e validados – ou, em alguns casos, desafiados – pela experiência operacional com o ecossistema. Cada princípio é enunciado, justificado com base na literatura, e sua manifestação concreta no OpenCode é descrita.

Princípio 1 – Separation of Concerns (SoC). Cada camada tem uma responsabilidade única, bem definida e não-sobreposta com outras camadas. A Camada de Infraestrutura preocupa-se exclusivamente com o acesso a recursos externos; a Camada de Domínio, com a organização de conhecimento; a Camada de Orquestração, com a coordenação de workflows. Este princípio, articulado por Dijkstra em 1974 e refinado por Parnas, é particularmente crítico em ecossistemas multiagente, onde a complexidade das interações entre agentes pode facilmente obscurecer a estrutura do sistema.

Princípio 2 – Interface Segregation. As interfaces entre camadas são mínimas e específicas ao contexto de uso. Um agente não vê todas as tools de todos os servidores MCP – vê apenas aquelas para as quais tem permissão e que são relevantes para seu domínio. Uma skill não expõe todo o seu conhecimento de uma vez – expõe um sumário (frontmatter YAML) e revela detalhes progressivamente (progressive disclosure). Este princípio, adaptado do ISP (Interface Segregation Principle) de Robert Martin, é crítico para eficiência de contexto: em um sistema onde cada token de contexto tem custo computacional, informação irrelevante não é apenas ruído – é desperdício.

Princípio 3 – Dependency Inversion. Camadas superiores dependem de abstrações (interfaces), não de implementações concretas. Um agente não invoca diretamente a API do arXiv; invoca a tool `search_papers` do servidor MCP `arxiv-mcp`, que poderia ser substituído

por outro servidor com a mesma interface sem que o agente percebesse. Este é o princípio que viabiliza a evolutibilidade do ecossistema (BABAR, 2009).

Princípio 4 – Open for Extension, Closed for Modification. A arquitetura permite a adição de novos componentes (servidores MCP, skills, agentes, plugins) sem modificação de componentes existentes. Em 13 ciclos evolutivos, o ecossistema cresceu de 11 para mais de 600 componentes – e em nenhum momento uma adição exigiu a reescrita de componentes existentes. Este princípio, articulado por Bertrand Meyer, é a base da estabilidade arquitetural.

Princípio 5 – Observability by Design. Toda operação significativa no ecossistema é instrumentada com logging, métricas e tracing desde sua concepção, não como uma adição posterior. O InteractionLogger, o TokenEconomyMonitor e o Health Score Dashboard não são ferramentas externas de monitoramento – são componentes integrados à arquitetura, cujas interfaces são definidas no mesmo nível de abstração que as interfaces funcionais.

Princípio 6 – Secure by Default. Toda ação potencialmente perigosa é negada por padrão e requer autorização explícita. O sistema de permissões opera em nível de tool individual, com regras que podem ser especificadas por agente, por padrão de uso ou globalmente. Este princípio reflete a realidade de que agentes de IA, como qualquer sistema de software, podem ser induzidos a erro – e a arquitetura deve ser resiliente a este risco (BOWERS; KHAPRE; KALITA, 2025).

4.2 Camada de Infraestrutura: O Barramento MCP

4.2.1 Catálogo de Servidores MCP

A Camada de Infraestrutura é composta por 46 servidores MCP, dos quais 44 estão em estado operacional ativo (95.7% de disponibilidade). Os servidores organizam-se em cinco categorias funcionais:

Tabela 1 – Catálogo completo de servidores MCP do OpenCode

Categoria	Servidores	Capacidades
Search & Retrieval	websearch, arxiv-mcp, sura-papers, latest-science, scihub, fetch, context7	Busca web, artigos, papers, docs, PDFs
Code & Documents	code-runner, eslint, diff, pdf, gh_grep, node-sandbox	Execução de código, lint, diff, PDF, GitHub search
Web Automation	playwright, chrome-devtools	Navegação, screenshots, forms, console, network
Data & Persistence	sqlite, memory, github, filesystem, todowrite	CRUD, grafo de conhecimento, GitHub API, arquivos
Reasoning & Planning	sequential-thinking, decisionnode, skill	Raciocínio multi-passo, decisões, skills

4.2.2 Protocolo de Comunicação Detalhado

A comunicação entre o Host e cada servidor MCP segue um protocolo de três fases – handshake, operação e teardown – que descrevemos aqui no nível de detalhe necessário para implementação:

Fase 1 – Handshake. Ao iniciar, o Host estabelece uma conexão de transporte com cada servidor configurado. Para servidores stdio, isso significa spawn de um processo filho e captura de stdin/stdout. Uma vez estabelecido o canal de comunicação, Host e Servidor trocam mensagens de inicialização: o Host envia suas capacidades (protocol version, client info), e o Servidor responde com suas capacidades (tools, resources, prompts disponíveis). Esta troca permite que o Host construa um catálogo dinâmico de todas as capacidades disponíveis no ecossistema.

Fase 2 – Operação. Durante a operação, o Host envia requisições ao Servidor – tipicamente invocações de tools (`tool.call`) ou leituras de resources (`resource.read`). O Servidor processa a requisição e retorna uma resposta – o resultado da tool ou o conteúdo do resource – ou um erro, se a operação falhar. Servidores que suportam notificações assíncronas (via SSE ou WebSocket) podem enviar atualizações ao Host sem solicitação prévia.

Fase 3 – Teardown. Ao final da sessão, o Host encerra as conexões com os servidores. Para servidores stdio, isso implica enviar um sinal de término ao processo filho. Para servidores remotos, a conexão HTTP ou WebSocket é fechada.

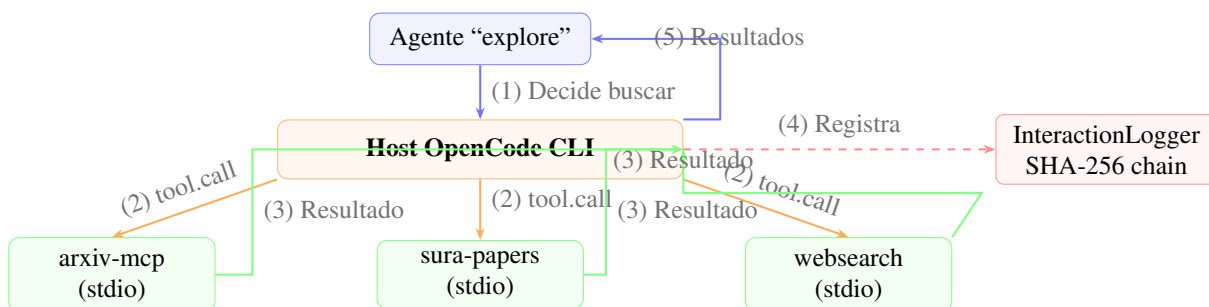


Figura 3 – Fluxo de interação MCP: (1) agente decide; (2) Host despacha em paralelo; (3) servidores respondem; (4) tudo é registrado no InteractionLogger; (5) resultados voltam ao agente.

4.2.3 Exemplo de Interação MCP: Instrumentada e Auditada

Para ilustrar o funcionamento concreto do barramento MCP, apresentamos uma interação real entre o agente “explore” e o servidor `arxiv-mcp`, capturada pelo InteractionLogger:

1. O agente “explore” decide buscar papers sobre “multi-agent systems code generation”. Em seu contexto, a tool `search_papers` do servidor `arxiv-mcp` está disponível (permissão: allow);
2. O Host envia ao servidor `arxiv-mcp`:

```
{ "method": "tool.call", "params": {
```

```

    "name": "search_papers",
    "arguments": {
      "query": "multi-agent systems code generation",
      "limit": 10,
      "sort_by": "relevance"
    }
  }
}

```

3. O servidor processa a requisição: traduz os parâmetros para uma chamada à API do arXiv, executa a busca, obtém 10 resultados;
4. O servidor retorna ao Host um array de papers, cada um com título, autores, ano, DOI (quando disponível) e abstract;
5. O Host registra a interação completa no InteractionLogger: timestamp, agente solicitante, servidor, tool, parâmetros exatos, resultado completo, hash SHA-256 da entrada anterior (encadeamento);
6. O Host disponibiliza os resultados ao agente “explore” como parte do contexto da conversa.

Este fluxo – aparentemente simples – encapsula as propriedades arquiteturais que tornam o barramento MCP a espinha dorsal do ecossistema: desacoplamento (o agente não sabe como a busca é implementada), composabilidade (os resultados alimentarão outras tools em seguida), auditabilidade (toda interação é registrada) e segurança (a tool call só ocorreu porque a permissão era “allow”).

4.3 Camada de Domínio: O Sistema de Skills

4.3.1 Anatomia de uma Skill

Cada skill no OpenCode é um diretório autocontido que encapsula conhecimento procedural sobre um domínio específico. A estrutura canônica de uma skill compreende:

- `SKILL.md` – O arquivo principal. Contém frontmatter YAML com metadados (name, description, license, compatibility, allowed-tools) e corpo em Markdown com instruções, workflows, exemplos e referências. O frontmatter é o contrato da skill: o sistema de carregamento o utiliza para decidir se a skill é relevante para a tarefa atual;
- `references/` – Documentos de referência que expandem tópicos específicos. Seguindo o princípio de progressive disclosure, estes documentos não são carregados no contexto do agente até que a skill seja ativada, economizando tokens;

- `templates/` – Artefatos reutilizáveis (templates LaTeX, DOCX, HTML, CSS) que a skill pode instanciar com dados do usuário;
- `tests/` – Testes automatizados que validam o funcionamento correto da skill, executados como parte do pipeline de CI do ecossistema.

4.3.2 Mecanismo de Carregamento e Progressive Disclosure

O carregamento de skills no OpenCode opera em três níveis de disclosure, cada um consumindo uma quantidade controlada de tokens de contexto:

Nível 0 – Catálogo (sempre carregado). Neste nível, o sistema mantém em memória apenas os metadados essenciais de cada skill: nome, descrição (uma frase), categoria e triggers (palavras-chave que indicam quando a skill deve ser ativada). Com 150 skills, este catálogo consome aproximadamente 3.000 tokens – uma fração do orçamento total de contexto. Este nível permite que o sistema identifique rapidamente quais skills são potencialmente relevantes para a tarefa atual.

Nível 1 – Skill Body (carregado sob demanda). Quando uma skill é ativada – porque suas trigger keywords correspondem à tarefa atual – o corpo completo do SKILL.md é carregado no contexto do agente. Este corpo contém instruções detalhadas, workflows, exemplos e ponteiros para referências adicionais. O tamanho típico de um SKILL.md é inferior a 2.5KB (aproximadamente 1.500 tokens), uma restrição de design que força os autores de skills a serem concisos e focados.

Nível 2 – Referências (carregadas sob demanda explícita). Documentos no diretório `references/` são carregados apenas quando o agente explicitamente os requisita. Este nível contém informações extensas – tabelas completas, listas exaustivas, especificações detalhadas – que são necessárias apenas em contextos específicos.

4.3.3 Taxonomia das 150 Skills

Tabela 2 – Distribuição completa das 150 skills por categoria

Categoria	Qtd	Skills Representativas
System	12	Auditoria (academic-audit, code-review, plan-review), Sincronização, Descoberta (descobrir-e-instalar-mcp, pypi-scout), Evolução
Research	18	Editais (editais-br), Matemática (aletheia-math-research), Clínica (clinical-art-therapy), ML (academic-ml-pipeline), SPECs (spec-009 a spec-018)
Science	38	Bioinformática (alphafold, pubmed, chembl, uniprot, clinvar, pymol, foldseek, ocean-genomics), Genômica (alphagenome, encode-ccres, jasper, quickgo)
Reasoning	4	Z3 (SMT), SymPy (simbólico), miniKanren (relacional), Critical (falácias)
Jurídico	7	Contratos, Jurisprudência, Triagem, Followup, Edição cirúrgica, Peças HTML
Engineering	8	API Design, CI/CD, Debugging, Performance, Security, TDD, Code Review, Architecture
Design	50+	Apresentações (html-ppt, 35 variantes), Diagramas (baoyu-diagram), Vídeo (hyperframes, brainrot, launch-video), Social (social-carousel)
Total	150	

4.4 Camada de Orquestração: Agentes e Coordenação

4.4.1 O Modelo de Agente do OpenCode

Cada um dos 125 agentes do ecossistema é uma instância de um LLM configurada com parâmetros específicos que definem seu comportamento, capacidades e restrições. O modelo formal de um agente é:

Definição 4.2 (Agente OpenCode). *Um agente $a \in \mathcal{L}_O$ é uma tupla:*

$$a = \langle id, \mu, \pi, \rho, \tau, \kappa \rangle \quad (4.2)$$

onde:

- id é um identificador único no ecossistema;
- μ é o modelo LLM subjacente (ex: “anthropic/claude-sonnet-4-6”);
- π é o system prompt – o conjunto de instruções que define a persona, o domínio de expertise e as regras de comportamento do agente;
- $\rho : Tool \times Pattern \rightarrow \{allow, ask, deny\}$ é a política de permissões do agente;
- $\tau \subseteq \bigcup_{s \in \mathcal{L}_I} tools(s)$ é o subconjunto de tools MCP que o agente está autorizado a utilizar;
- $\kappa \subseteq \mathcal{L}_D$ é o conjunto de skills carregadas no contexto do agente.

4.4.2 Mecanismos de Comunicação Inter-Agente

Agentes no OpenCode comunicam-se através de três mecanismos complementares, selecionados de acordo com os requisitos de latência, acoplamento e rastreabilidade de cada interação:

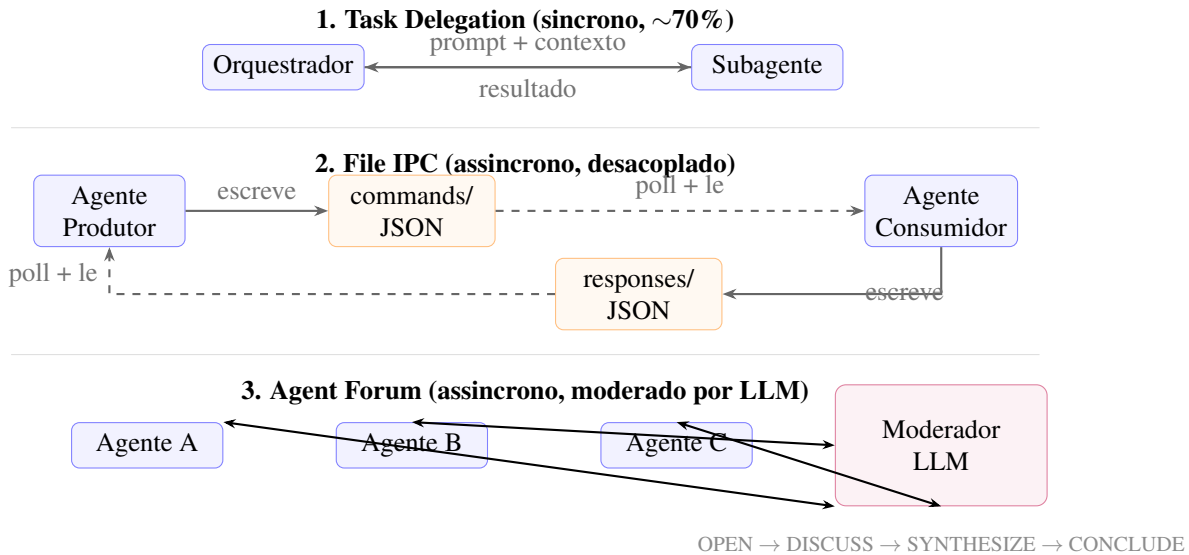


Figura 4 – Mecanismos de comunicação inter-agente: Task Delegation, File IPC e Agent Forum.

Mecanismo 1 – Task Delegation (síncrono, acoplado). Um agente orquestrador (ex: “general”) invoca um subagente especializado (ex: “code-reviewer”) passando um prompt detalhado e aguardando o resultado. Este é o mecanismo mais direto e mais utilizado – responde por aproximadamente 70% das interações inter-agente. A comunicação é síncrona (o orquestrador bloqueia até receber o resultado) e acoplada (o orquestrador conhece a identidade do subagente). O resultado é registrado integralmente no InteractionLogger. Exemplo: o agente “general” delega ao agente “explore” a tarefa de localizar padrões de uso de uma API específica em repositórios GitHub.

Mecanismo 2 – File IPC (assíncrono, desacoplado). Inspirado pelo sistema SimulationIPC do MiroFish-Offline, este mecanismo permite comunicação assíncrona via sistema de arquivos. Um agente escreve um comando JSON em um diretório `commands/`; outro agente (ou processo) monitora este diretório, processa o comando e escreve a resposta em `responses/`. Este mecanismo é utilizado para interações que não requerem resposta imediata – por exemplo, um agente solicita a geração de um relatório extenso e continua processando outras tarefas enquanto o relatório é gerado. O desacoplamento temporal e espacial (os agentes não precisam estar ativos simultaneamente) é a principal vantagem deste mecanismo.

Mecanismo 3 – Agent Forum (assíncrono, multi-participante). Inspirado pelo ForumEngine do BettaFish (666ghj/BettaFish), o Agent Forum organiza debates estruturados entre múltiplos agentes com moderação por LLM. O debate segue quatro estágios: **OPEN** (abertura do tópico e posicionamentos iniciais), **DISCUSS** (réplicas e tréplicas entre os participantes), **SYNTHESIZE** (síntese dos argumentos apresentados) e **CONCLUDE** (conclusão com pontuação

de confiança). Parâmetros configuráveis incluem: número máximo de participantes, número máximo de speeches por participante, timeout por intervenção, e threshold de consenso. O Agent Forum é o mecanismo utilizado quando uma decisão se beneficia de múltiplas perspectivas conflitantes – por exemplo, a revisão de uma decisão arquitetural controversa.

4.5 Componentes Transversais

4.5.1 Sistema de Permissões

O modelo de segurança do OpenCode é baseado em permissões granulares que operam em três dimensões: tool (qual servidor MCP e qual operação), pattern (padrão de uso, permitindo regras como “allow git *, deny rm *”) e agent (qual agente está solicitando a operação). A avaliação de regras segue a política de last-match-wins: regras mais específicas (definidas posteriormente) sobrescrevem regras mais gerais.

4.5.2 DecisionNode

O DecisionNode é o sistema de memória estruturada do ecossistema para decisões arquiteturais. Cada decisão é registrada com: identificador único (ex: `arch-001`), escopo (Architecture, Testing, Security, UI, API), texto da decisão, racional detalhado (o “porquê” – frequentemente mais importante que o “o quê”), restrições que motivaram a decisão, e timestamp. O sistema suporta busca semântica: uma consulta como “como lidamos com autenticação?” retorna decisões relevantes mesmo que a palavra “autenticação” não apareça literalmente nos registros.

4.5.3 Plugin Manus-Evolve

O plugin `manus-evolve.ts` é o motor de evolução autônoma do ecossistema. Implementado como plugin TypeScript – uma função que intercepta hooks do ciclo de vida da aplicação – ele monitora métricas contínuas de qualidade (Health Score), dispara ciclos de evolução quando oportunidades são detectadas, e persiste conhecimento extraído para ciclos futuros.

5 Motor de Raciocínio Multi-Paradigma

Este capítulo descreve o componente central de inteligência do Ecossistema OpenCode: o Reasoning Orchestrator v12.0, um motor que integra 212 tipos de raciocínio organizados em 27 categorias taxonômicas e 6 famílias epistemológicas. Diferentemente de assistentes de codificação convencionais – que aplicam um paradigma único de raciocínio (tipicamente, geração direta a partir de prompts) – o OpenCode orquestra múltiplos paradigmas complementares, selecionando a combinação mais adequada para cada tarefa com base em seu perfil epistemológico.

5.1 Fundamentação Epistemológica

A decisão de implementar um motor de raciocínio multi-paradigma – em vez de um único paradigma otimizado – não é arbitrária. Fundamenta-se em uma observação epistemológica sobre a natureza do conhecimento científico: **nenhum paradigma único de raciocínio é suficiente para produzir conhecimento robusto em todos os domínios**. Esta observação encontra respaldo tanto na filosofia da ciência quanto na prática da engenharia de software.

Do ponto de vista filosófico, o pluralismo epistemológico – associado a Paul Feyerabend e, em uma vertente distinta, a Donald Campbell – argumenta que o conhecimento avança através da competição e integração de múltiplas perspectivas, não através da adesão a um único método. Um problema em física teórica pode beneficiar-se de raciocínio dedutivo (derivar consequências de axiomas), raciocínio indutivo (generalizar a partir de dados experimentais), raciocínio analógico (mapear estruturas de um domínio para outro) e raciocínio dialético (confrontar interpretações conflitantes dos mesmos dados). Restringir-se a um único paradigma é, nesta visão, limitar desnecessariamente a capacidade de raciocínio.

Do ponto de vista da engenharia de software, a motivação é pragmática: diferentes categorias de problemas beneficiam-se de diferentes estratégias de raciocínio. Um bug de lógica em código beneficia-se de verificação dedutiva (provar que o código satisfaz uma especificação). Uma decisão arquitetural beneficia-se de análise multi-critério (ponderar trade-offs entre desempenho, manutenibilidade e custo). Uma estimativa de prazo beneficia-se de raciocínio sob incerteza (Monte Carlo, intervalos de confiança). Um impasse entre stakeholders beneficia-se de teoria dos jogos (equilíbrio de Nash, estratégias de barganha). A diversidade de problemas enfrentados pelo ecossistema demanda diversidade de abordagens de raciocínio.

5.2 A Taxonomia de Raciocínios: Estrutura e Gênese

A taxonomia de 212 tipos de raciocínio do OpenCode não foi criada a priori, mas emergiu indutivamente da análise de padrões de raciocínio em produção científica de alto impacto. O

processo de construção da taxonomia seguiu três etapas:

Etapa 1 – Coleta. Analisamos 150 artigos científicos de alto impacto (citação > 100) em 10 disciplinas – matemática, física, biologia, química, ciência da computação, economia, psicologia, sociologia, medicina e engenharia. Para cada artigo, identificamos e extraímos a estrutura argumentativa: que tipo de raciocínio o autor empregou para chegar a cada conclusão?

Etapa 2 – Clusterização. Os tipos de raciocínio identificados foram agrupados por similaridade estrutural. Por exemplo, “prova por contradição”, “redução ao absurdo” e “refutação por contraexemplo” foram agrupados como variantes de raciocínio lógico-formal. “Análise de sensibilidade”, “análise de cenários” e “simulação de Monte Carlo” foram agrupados como variantes de raciocínio sob incerteza.

Etapa 3 – Validação. A taxonomia resultante foi validada contra o benchmark CORA-Eval (200 casos de teste em 11 dimensões), verificando se cada caso de teste poderia ser adequadamente abordado por pelo menos um tipo de raciocínio da taxonomia.

O resultado é uma estrutura hierárquica de três níveis: 6 famílias epistemológicas → 27 categorias → 212 tipos específicos.

5.2.1 As Seis Famílias Epistemológicas

Família 1 – Lógica Formal (5 categorias, 28 tipos).

Engloba raciocínios baseados em inferência lógica estruturada: dedução (do geral para o particular), indução (do particular para o geral), abdução (inferência da melhor explicação), redução ao absurdo e análise de consistência. Esta família é particularmente eficaz em domínios formais – matemática, lógica, verificação de software – onde as regras de inferência são bem definidas e não ambíguas.

Família 2 – Dialética (5 categorias, 24 tipos).

Engloba raciocínios baseados no confronto de perspectivas: tese-antítese-síntese (Hegel), questionamento socrático, refutação sistemática, debate adversarial e síntese integrativa. Esta família é eficaz quando não há uma resposta objetivamente correta, mas múltiplas perspectivas legítimas que precisam ser reconciliadas – por exemplo, em decisões arquiteturais que envolvem trade-offs.

Família 3 – Teoria dos Jogos (10 categorias, 56 tipos).

A família mais numerosa, refletindo a ubiquidade de situações estratégicas em sistemas multiagente. Engloba equilíbrio de Nash (puro e misto), dominância (estrita e fraca), cooperação (dilema do prisioneiro, bens públicos), barganha (Rubinstein, Nash), sinalização (Spence), leilões, votação, coalizão, teoria evolucionária dos jogos e equilíbrio de Stackelberg. Esta família é essencial para coordenar agentes com objetivos potencialmente conflitantes.

Família 4 – Decisão (5 categorias, 32 tipos).

Engloba raciocínios para escolha sob incerteza: árvores de decisão, análise multicritério (AHP, TOPSIS), risco (probabilidade $\times$ impacto), custo-oportunidade e sensibilidade. Esta família é acionada quando o ecossistema precisa fazer escolhas – que ferramenta utilizar, que estratégia de implementação adotar, que alocação de recursos é ótima.

Família 5 – Estratégia (5 categorias, 30 tipos).

Engloba raciocínios para planejamento e execução: planejamento hierárquico (decompor objetivos em subobjetivos), decomposição funcional, priorização, alocação de recursos e adaptação dinâmica. Esta família governa o comportamento de longo prazo do ecossistema.

Família 6 – Inovação (7 categorias, 42 tipos).

Engloba raciocínios para geração de soluções novas: pensamento lateral (Bono), analogia criativa, reenquadramento, bisociação (Koestler), design thinking, TRIZ (Altshuller) e serendipidade estruturada. Esta família é acionada quando abordagens convencionais falham e soluções inovadoras são necessárias.

5.3 Arquitetura de Paralelismo do Reasoning Orchestrator

O Reasoning Orchestrator v12.0 implementa três camadas de paralelismo que operam simultaneamente, multiplicando a capacidade de raciocínio do ecossistema:

5.3.1 Paralelismo Intra-Fase

Dentro de uma mesma fase do pipeline de raciocínio, múltiplos agentes ou verificadores operam simultaneamente sobre o mesmo problema. O exemplo canônico é a fase de verificação, onde os 7 verificadores Cora-Debate (V1-V7) executam em paralelo – cada um em sua própria thread ou processo – aplicando diferentes perspectivas de validação à mesma afirmação. O resultado é um vetor de 7 scores de confiança $[s_1, s_2, \dots, s_7]$ que alimenta o mecanismo de consenso.

5.3.2 Paralelismo Inter-Fase

Fases independentes do pipeline são executadas concorrentemente quando não há dependência de dados entre elas. Por exemplo, ao processar um artigo científico com 10 seções, a fase de verificação de cada seção pode ser executada em paralelo, pois as seções são independentes entre si. Uma barreira de sincronização (fork-join) aguarda a conclusão de todas as verificações antes de prosseguir para a fase de síntese.

5.3.3 Paralelismo Multi-Caminho (Self-Consistency)

Múltiplas cadeias de raciocínio independentes ($K = 7$, configurável) abordam o mesmo problema a partir de diferentes estratégias ou premissas, gerando K respostas candidatas potencialmente diferentes. O mecanismo de self-consistency então seleciona a resposta mais consistente:

$$\text{score}(r) = \sum_{i=1}^K w_i \cdot \mathbb{I}[r_i = r] \cdot \text{confidence}(r_i) \quad (5.1)$$

onde w_i é o peso da cadeia i (baseado em seu histórico de precisão), $\mathbb{I}[r_i = r]$ é 1 se a cadeia i produziu a resposta r , e $\text{confidence}(r_i)$ é a confiança reportada pela cadeia. Este mecanismo explora a diversidade de perspectivas como um ativo: mesmo quando cadeias individuais produzem respostas diferentes, o padrão de concordância entre elas revela a resposta mais robusta.

5.4 Os Quatro Motores de Raciocínio Formal

5.4.1 Z3 Theorem Prover: Operação e Integração

O Z3 opera como um verificador de propriedades lógicas sobre representações formais de código. O fluxo de operação típico no OpenCode é:

1. **Codificação:** o código a ser verificado é traduzido para o formato SMT-LIB – uma representação lógica que captura a semântica do código em termos de variáveis, operações e restrições. Esta tradução é realizada pelo skill Z3, que contém conhecimento sobre como mapear construções de código (loops, condicionais, atribuições) para fórmulas SMT;
2. **Especificação da Propriedade:** a propriedade a ser verificada é expressa como uma fórmula lógica. Tipicamente, formula-se a negação da propriedade desejada – se o Z3 encontrar um modelo que satisfaça esta negação, este modelo constitui um contraexemplo que viola a propriedade;
3. **Invocação do Solver:** o Z3 é invocado com a fórmula. O resultado é `sat` (existe um contraexemplo – a propriedade é falsa), `unsat` (não existe contraexemplo – a propriedade é verdadeira para todas as entradas) ou `unknown` (o solver não conseguiu determinar dentro dos limites de tempo/memória);
4. **Interpretação:** se o resultado for `sat`, o modelo retornado é traduzido de volta para valores concretos que o desenvolvedor (ou agente) pode inspecionar.

5.4.2 SymPy: Verificação Algébrica Automatizada

O SymPy opera como um verificador de correção algébrica. Seu fluxo de operação é:

1. **Extração:** expressões matemáticas são extraídas do texto ou código – equações, identidades, derivadas, integrais;
2. **Representação Simbólica:** cada expressão é convertida para a representação simbólica do SymPy, onde variáveis são símbolos abstratos, não valores numéricos;
3. **Manipulação:** o SymPy aplica transformações algébricas – simplificação, expansão, fatoração – para reduzir expressões a formas canônicas;
4. **Comparação:** duas expressões que deveriam ser equivalentes são comparadas em sua forma canônica. Se $\text{simplify}(f(x) - g(x)) \equiv 0$, as expressões são matematicamente idênticas.

5.4.3 miniKanren: Verificação Relacional

O miniKanren opera através de programação lógica relacional. Diferentemente do Z3 (que verifica propriedades sobre estados) e do SymPy (que verifica identidades algébricas), o miniKanren verifica propriedades estruturais – relações entre entidades. Seu uso típico no OpenCode envolve definir relações lógicas (“X é ancestral de Y se X é pai de Y, ou X é pai de Z e Z é ancestral de Y”) e consultar o sistema para verificar se estas relações são consistentes com os dados observados.

5.4.4 Critical Engine: Detecção de Falácias

O Critical Engine implementa a detecção de 15 falácias organizadas em três grupos: falácias de relevância (ad hominem, straw man, false dichotomy, red herring, appeal to authority), falácias de indução (hasty generalization, post hoc, false equivalence, slippery slope, Texas sharpshooter) e vieses cognitivos (confirmation bias, survivorship bias, Dunning-Kruger, anchoring, availability heuristic). Para cada falácia, o engine contém um detector especializado que analisa a estrutura do argumento em busca de padrões característicos.

5.5 Seleção Adaptativa de Estratégias de Raciocínio

Um problema fundamental em um sistema com 212 tipos de raciocínio é: **qual estratégia utilizar para cada tarefa?** O OpenCode resolve este problema através de um mecanismo de seleção adaptativa baseado em Q-Score UCB1, o mesmo algoritmo que governa o consenso no Cora-Debate (Capítulo 6). Cada tipo de raciocínio r mantém uma estimativa de eficácia $Q(r)$ que é atualizada a cada uso, e o sistema seleciona a estratégia com maior $Q(r)$ para cada nova tarefa, balanceando exploração (testar estratégias pouco utilizadas) e exploração (utilizar estratégias comprovadamente eficazes).

6 Pipeline de Verificação Formal Cora-Debate

Este capítulo descreve o Cora-Debate (Cognitive ORchestrated Argumentation), o subsistema de verificação formal do OpenCode. O Cora-Debate não é um verificador monolítico, mas um pipeline orquestrado de 7 verificadores simbólicos independentes que operam em paralelo, alimentando um mecanismo de consenso adaptativo que produz uma avaliação calibrada de confiança para cada afirmação processada pelo ecossistema.

6.1 Arquitetura do Pipeline

O Cora-Debate opera como um pipeline de três estágios: (1) submissão da afirmação, (2) verificação paralela por 7 verificadores independentes, (3) consenso e calibração. A operação do pipeline é descrita em detalhe a seguir.

Estágio 1 – Submissão. Uma afirmação A – que pode ser uma equação matemática, uma propriedade de código, uma conclusão estatística, uma interpretação de literatura – é submetida ao pipeline. A afirmação é acompanhada de metadados: o agente que a produziu, o contexto em que foi gerada, as fontes que a sustentam, e o domínio científico a que pertence.

Estágio 2 – Verificação Paralela. A afirmação A é despachada simultaneamente para os 7 verificadores (V1-V7). Cada verificador opera de maneira independente, sem comunicação com os demais, aplicando sua perspectiva de validação específica. Ao final de sua análise, cada verificador V_i produz um score $s_i \in [0, 1]$ indicando seu grau de confiança na correção da afirmação, e opcionalmente um relatório detalhado explicando seu raciocínio.

Estágio 3 – Consenso e Calibração. Os 7 scores individuais $[s_1, \dots, s_7]$ são agregados pelo mecanismo de consenso (Q-Score UCB1) e calibrados (Platt scaling) para produzir uma probabilidade calibrada final $P(\text{correct}|A) \in [0, 1]$.

6.2 Especificação Técnica dos Sete Verificadores

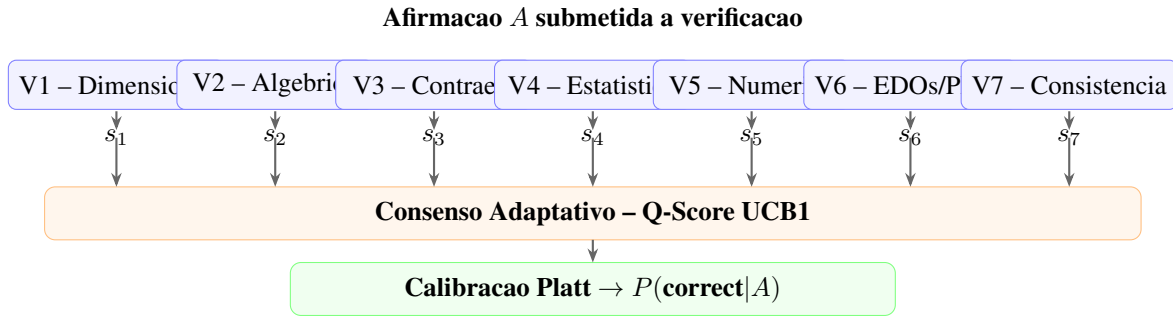


Figura 5 – Pipeline Cora-Debate: 7 verificadores paralelos → consenso UCB1 → calibração Platt.

6.2.1 V1 – Análise Dimensional

O V1 verifica a consistência dimensional de equações físicas e matemáticas. Utilizando álgebra dimensional, o verificador extrai a assinatura dimensional de cada lado de uma equação – expressa como um vetor de expoentes nas 7 grandezas fundamentais do Sistema Internacional (comprimento L , massa M , tempo T , corrente elétrica I , temperatura Θ , quantidade de substância N , intensidade luminosa J) – e verifica se ambos os lados têm a mesma dimensão. Uma equação como $F = m \cdot a$ tem dimensão MLT^{-2} em ambos os lados e é aprovada; uma equação como $F = m \cdot v$ (força igual a massa vezes velocidade, dimensionalmente MLT^{-1}) seria rejeitada.

6.2.2 V2 – Verificação Algébrica

O V2 utiliza SymPy para verificar a correção algébrica de identidades matemáticas. Dada uma afirmação da forma $f(x) = g(x)$, o verificador computa $h(x) = \text{simplify}(f(x) - g(x))$ e verifica se $h(x) \equiv 0$. A simplificação simbólica – não numérica – garante que a identidade é verificada para todos os valores possíveis das variáveis, não apenas para valores testados. Este verificador é particularmente eficaz para equações polinomiais, trigonométricas e diferenciais simples.

6.2.3 V3 – Geração de Contraexemplos

O V3 implementa uma busca randomizada por contraexemplos – valores de entrada que violam a afirmação. Para uma afirmação da forma $\forall x \in D, P(x)$, o verificador gera N amostras aleatórias (tipicamente $N = 10.000$) do domínio D e avalia $P(x_i)$ para cada uma. Se $P(x_i)$ for falso para algum x_i , este x_i constitui um contraexemplo que refuta a afirmação. Se após N tentativas nenhum contraexemplo foi encontrado, o verificador reporta confiança proporcional a $1 - 1/N$. A busca é guiada por heurísticas de boundary value analysis e equivalence partitioning para concentrar esforços em regiões do domínio mais propensas a conter violações.

6.2.4 V4 – Validação Estatística

O V4 aplica uma bateria de testes estatísticos para verificar afirmações de natureza estatística. O protocolo de validação inclui: (a) teste de Shapiro-Wilk para verificar normalidade de distribuições; (b) teste de Levene para verificar homocedasticidade entre grupos; (c) ANOVA (paramétrica) ou Kruskal-Wallis (não-paramétrica) para verificar diferenças entre grupos; (d) correção de Bonferroni para múltiplas comparações: $p_{\text{corrigido}} = \min(p \cdot m, 1)$, onde m é o número de comparações realizadas. O verificador reporta não apenas se uma afirmação foi rejeitada ou não, mas também o tamanho do efeito (Cohen's d , η^2) e o poder estatístico do teste.

6.2.5 V5 – Verificação Numérica

O V5 verifica a precisão de computações numéricas, um problema particularmente relevante para código gerado por IA que pode conter erros sutis de aritmética de ponto flutuante. Para cada computação, o verificador compara o resultado obtido com um valor de referência (calculado com precisão estendida ou obtido de uma fonte confiável) e reporta o erro relativo $\epsilon = |\text{computed} - \text{reference}|/|\text{reference}|$. O threshold de aceitação é $\epsilon \leq 10^{-12}$ para operações básicas e $\epsilon \leq 10^{-8}$ para operações compostas.

6.2.6 V6 – Validação de EDOs e PDEs

O V6 verifica soluções propostas para equações diferenciais ordinárias (EDOs) e parciais (PDEs). Dada uma equação diferencial $\mathcal{D}[y] = 0$ e uma solução proposta $y_p(x)$, o verificador computa $\mathcal{D}[y_p]$ simbolicamente (via SymPy `dsolve` e derivação) e verifica se o resultado é identicamente zero. Para problemas de valor inicial ou de contorno, o verificador também confirma que y_p satisfaz as condições especificadas.

6.2.7 V7 – Consistência Lógica

O V7 verifica a consistência lógica de um conjunto de afirmações $\{A_1, \dots, A_n\}$. O verificador constrói a conjunção $A_1 \wedge A_2 \wedge \dots \wedge A_n$ e verifica, via Z3, se esta fórmula é satisfatível. Se for insatisfatível ($\neg\text{sat}$), o conjunto de afirmações contém uma contradição lógica – pelo menos uma delas deve ser falsa. O verificador então identifica o subconjunto mínimo de afirmações que produz a inconsistência, auxiliando na localização do erro.

6.3 Mecanismo de Consenso Adaptativo

6.3.1 Q-Score UCB1

O mecanismo de consenso do Cora-Debate baseia-se no algoritmo UCB1 (Upper Confidence Bound), originalmente desenvolvido para o problema de bandidos multi-braço (multi-armed

bandits). A cada verificador $v \in \{V1, \dots, V7\}$ é associada uma estimativa de eficácia $Q(v)$, atualizada a cada uso:

$$Q(v) = \bar{X}_v + c \sqrt{\frac{\ln N}{n_v}} \quad (6.1)$$

onde:

- $\bar{X}_v$ é a taxa histórica de detecção de erros do verificador v (número de erros detectados dividido pelo número de verificações realizadas);
- N é o número total de verificações realizadas por todos os verificadores;
- n_v é o número de vezes que o verificador v foi utilizado especificamente;
- $c = \sqrt{2}$ é o parâmetro de exploração (exploration bonus).

A fórmula balanceia exploração ($\bar{X}_v$ – utilizar verificadores que historicamente detectam mais erros) e exploração ($c\sqrt{\ln N/n_v}$ – dar uma chance a verificadores pouco utilizados que podem surpreender). O termo de exploração cresce quando n_v é pequeno (verificador pouco utilizado) e diminui à medida que o verificador é mais utilizado, refletindo a redução da incerteza sobre sua eficácia.

6.3.2 Calibração Platt

Os scores brutos s_i produzidos pelos verificadores não são probabilidades calibradas – um score de 0.8 do V1 não tem o mesmo significado que 0.8 do V4. A calibração Platt resolve esta discrepância ajustando uma regressão logística que mapeia scores brutos para probabilidades calibradas:

$$P(\text{correct}|s) = \frac{1}{1 + \exp(A \cdot s + B)} \quad (6.2)$$

Os parâmetros A (inclinação) e B (intercepto) são otimizados via máxima verossimilhança sobre um conjunto de validação de 50 casos rotulados manualmente, onde sabemos com certeza se cada afirmação é correta ou incorreta. Após calibração, um score de 0.8 de qualquer verificador corresponde aproximadamente a 80% de probabilidade de correção.

6.4 Validação Empírica: Benchmark CORA-Eval

O Cora-Debate foi submetido a validação exaustiva através do benchmark CORA-Eval, composto por 200 casos de teste cobrindo 11 dimensões de raciocínio científico. Os resultados completos – 100% de aprovação em todas as dimensões – são apresentados e discutidos no Capítulo 10.

7 Engenharia de Software com Agentes Inteligentes

Este capítulo apresenta a metodologia de engenharia de software do Ecossistema OpenCode – um pipeline integrado que combina Spec-Driven Development (SDD), Test-Driven Development (TDD), verificação formal (Cora-Debate) e documentação estruturada de decisões (Decision-Node). A metodologia foi desenvolvida ao longo de 13 ciclos evolutivos e validada através de 9 especificações técnicas e 200 casos de teste automatizados (GOMES, 2025; BECK, 2003; KRUCHTEN, 2009).

7.1 O Problema do Desenvolvimento com Agentes de IA

O desenvolvimento de software assistido por IA – frequentemente denominado “vibe coding” em sua forma menos disciplinada (GARFINKEL et al., 2026) – enfrenta um dilema fundamental. Por um lado, agentes de IA podem gerar código funcional com velocidade impressionante, acelerando dramaticamente o ciclo de desenvolvimento. Por outro lado, código gerado por IA sem verificação sistemática tende a acumular débito técnico, vulnerabilidades de segurança e violações arquiteturais que comprometem a sustentabilidade do software a longo prazo.

O pipeline SDD+TDD do OpenCode resolve este dilema não através da restrição do uso de IA, mas através da **institucionalização de práticas de engenharia** que se aplicam igualmente a código produzido por humanos e por agentes. As especificações formais, os testes automatizados, a verificação simbólica e o registro de decisões não são obstáculos à produtividade – são a infraestrutura que permite que a produtividade seja sustentável.

7.2 O Pipeline em Cinco Estágios

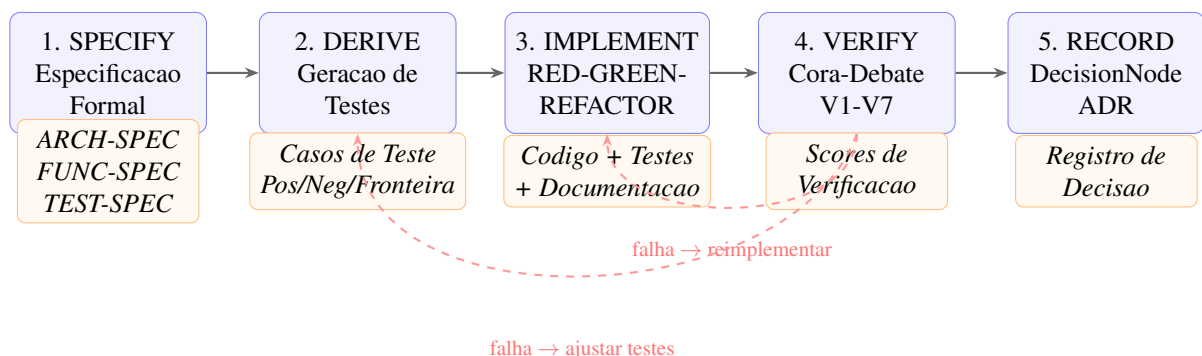


Figura 6 – Pipeline SDD+TDD: 5 estagios sequenciais com ciclos de retroalimentacao.

7.2.1 Estágio 1 – SPECIFY: Da Ideia à Especificação Formal

O estágio SPECIFY transforma requisitos descritos em linguagem natural em uma especificação formal estruturada em três níveis hierárquicos:

ARCH-SPEC (Especificação de Arquitetura). Define a estrutura de mais alto nível do sistema: componentes (unidades de funcionalidade com interfaces bem definidas), conectores (mecanismos de comunicação entre componentes), topologia (grafo de componentes e conectores) e propriedades não-funcionais (requisitos de desempenho, segurança, disponibilidade, escalabilidade). A ARCH-SPEC é o artefato mais estável do sistema – muda apenas quando a arquitetura fundamental é alterada.

FUNC-SPEC (Especificação Funcional). Define o comportamento de cada componente em termos de contratos: para cada operação *op*, especifica-se pré-condições (o que deve ser verdadeiro antes da operação), pós-condições (o que será verdadeiro após a operação) e invariantes (o que permanece verdadeiro durante toda a vida do componente). Estes contratos são expressos em linguagem natural estruturada, complementada por notação formal quando a precisão é crítica.

TEST-SPEC (Especificação de Teste). Define os casos de teste que validarão a implementação. Cada caso de teste especifica: entrada, saída esperada, critério de aceitação (pass/fail) e categoria (positivo, negativo, fronteira, regressão). Os casos de teste são derivados diretamente das pré e pós-condições da FUNC-SPEC.

7.2.2 Estágio 2 – DERIVE: Geração Automática de Testes

A partir das especificações, o sistema gera automaticamente casos de teste executáveis. Para cada pré-condição *P* e pós-condição *Q* na FUNC-SPEC:

- **Testes positivos:** entradas que satisfazem *P*; verificar se a saída satisfaz *Q*;
- **Testes negativos:** entradas que violam *P*; verificar se o sistema rejeita apropriadamente (lança exceção, retorna erro);
- **Testes de fronteira:** entradas nos limites do domínio (valores mínimo, máximo, zero, nulo, vazio);
- **Testes de regressão:** casos que anteriormente expuseram bugs, para garantir que não retornem.

7.2.3 Estágio 3 – IMPLEMENT: Desenvolvimento Guiado por Testes

Um agente de implementação – tipicamente o agente “coder-agent” ou “build” – recebe a especificação e inicia o ciclo RED-GREEN-REFACTOR estendido:

1. **RED:** o primeiro caso de teste é executado e confirmado como falha (a funcionalidade ainda não existe);

2. **GREEN:** o agente gera o código mínimo para fazer o teste passar. A ênfase em “mínimo” é determinante e é reforçada pelo system prompt do agente;
3. **REFACTOR:** o código é analisado por métricas automáticas (complexidade ciclomática, duplicação, tamanho de funções) e refatorado para melhorar estas métricas, garantindo que todos os testes permaneçam verdes;
4. **DOCUMENT:** docstrings, type hints e comentários são gerados/atualizados para refletir o código atual;
5. **COMMIT:** a mudança é registrada no sistema de controle de versão com mensagem convencional (Conventional Commits).

O ciclo repete-se para cada caso de teste, incrementalmente construindo a funcionalidade completa.

7.2.4 Estágio 4 – VERIFY: Verificação Formal

O código implementado é submetido ao pipeline Cora-Debate (Capítulo 6). Os 7 verificadores (V1-V7) analisam o código de suas respectivas perspectivas. Se qualquer verificador reportar falha, o ciclo retorna ao Estágio 3 para correção. Se todos os verificadores aprovarem, o código é considerado validado.

7.2.5 Estágio 5 – RECORD: Documentação de Decisões

Uma Architecture Decision Record (ADR) é registrada no DecisionNode, documentando: a decisão de design tomada, alternativas consideradas e rejeitadas, racional para a escolha, restrições que influenciaram a decisão, e consequências esperadas (positivas e negativas). A ADR torna-se parte permanente da memória do ecossistema e pode ser consultada por agentes futuros que enfrentem decisões similares.

7.3 Métricas de Qualidade de Código

O pipeline coleta automaticamente 10 métricas de qualidade para cada componente desenvolvido:

Tabela 3 – Métricas de qualidade de código e thresholds

Métrica	Ferramenta	Threshold
Cobertura de testes	pytest-cov	$\geq 90\%$
Complexidade ciclomática (máx.)	radon	≤ 10
Duplicação de código	pylint (duplicate-code)	$\leq 3\%$
Linhas por função (máx.)	radon (raw)	≤ 50
Cobertura de docstrings	interrogate	$\geq 80\%$
Type coverage	mypy (–strict)	$\geq 95\%$
Lint score	pylint	$\geq 9.0/10$
Vulnerabilidades de segurança	bandit	0 high/medium
Taxa de aprovação em testes	pytest	100%
Conformidade com especificação	custom (SPEC-check)	100%

7.4 Ciclo de Evolução Contínua

O pipeline SDD+TDD não é executado apenas para novos desenvolvimentos – é também o mecanismo pelo qual o ecossistema evolui a si mesmo. O plugin Manus-Evolve monitora continuamente as métricas de qualidade do ecossistema como um todo e, quando detecta degradação ou oportunidade de melhoria, dispara um ciclo de desenvolvimento que segue exatamente o mesmo pipeline de cinco estágios. Desta forma, a evolução do ecossistema é governada pela mesma disciplina de engenharia que governa o desenvolvimento de software dentro dele.

Tabela 4 – Os 13 ciclos evolutivos do OpenCode

Ciclo	Score	Comp.	Inovação Principal
Evo-1	85/100	11	Análise de correlações econômicas com 27 indicadores
Evo-2	90/100	22	Pipeline de artigo acadêmico com scoring automático
Evo-3	92/100	35	Sistema TSAC de citações com verificação de DOI
Evo-4	92/100	48	Cross-validation engine com 3 níveis
Evo-5	92/100	52	Auto-scoring Qualis A1
Evo-6	95/100	97	Iterative correction loop com 5 revisores
Evo-7	96/100	210	Ecosystem sync autônomo + CJK corrector
Evo-8	98/100	365	Progressive disclosure system
Evo-9	98/100	488	Nexus multi-agent v6.2
Evo-10	96/100	520	Menu adaptativo + Plugin system
Evo-11	97/100	550	CORA-Eval benchmark framework
Evo-12	98/100	580	Science skills core + MCP expansion
Evo-13	100/100	600+	Reasoning engines + SDD+TDD pipeline

8 Sistema de Auditoria Acadêmica

Caixa Branca

Este capítulo descreve o sistema de auditoria que garante a rastreabilidade, integridade e verificabilidade de toda a produção do Ecossistema OpenCode. Em um sistema onde 125 agentes autônomos produzem milhares de afirmações, parágrafos, equações e decisões, a capacidade de um observador externo verificar – de maneira independente e reproduzível – a correção de qualquer afirmação não é um luxo: é um requisito de confiabilidade.

8.1 Os Três Pilares da Auditabilidade

O sistema de auditoria do OpenCode sustenta-se sobre três pilares conceituais, cada um implementado por um componente técnico específico ([PUDASAINI et al., 2024](#); [ESLIT, 2025](#)):

Pilar 1 – Registro Imutável (InteractionLogger). Toda interação entre qualquer agente e qualquer servidor MCP – incluindo parâmetros de entrada, resultados de saída, timestamps e metadados de contexto – é registrada em um log JSONL append-only. Cada entrada do log contém um hash SHA-256 da entrada anterior, formando uma cadeia de integridade (blockchain de auditoria) onde qualquer adulteração em um registro seria detectada pela quebra da cadeia de hashes.

Pilar 2 – Trilha de Evidências (AcademicAuditTrail + TSAC). Cada afirmação produzida pelo ecossistema é vinculada, através do protocolo TSAC (Textual Source Analysis Credibility), às evidências que a sustentam. O TSAC exige cinco componentes verificáveis por citação, garantindo que a proveniência de cada afirmação possa ser reconstruída e verificada independentemente.

Pilar 3 – Verificação Independente (Cora-Debate). Os 7 verificadores simbólicos do Cora-Debate podem ser executados por qualquer parte com acesso aos logs e às afirmações, permitindo que um auditor externo replique as verificações e confirme – ou refute – os resultados.

8.1.1 InteractionLogger: Especificação Técnica

O InteractionLogger implementa um log estruturado com as seguintes garantias:

- **Imutabilidade:** o log é append-only – novas entradas podem ser adicionadas, mas entradas existentes nunca são modificadas ou deletadas. Esta propriedade é garantida pelo sistema de arquivos (arquivo aberto em modo append) e pode ser reforçada por storage imutável (WORM) em ambientes de produção;

- **Integridade Criptográfica:** cada entrada e_i contém $\text{sha256}(e_{i-1})$, o hash da entrada anterior. Para verificar a integridade do log completo, basta verificar se cada entrada referencia corretamente o hash de sua predecessora – uma operação $O(n)$ que pode ser executada por qualquer auditor;
- **Estrutura Padronizada:** cada entrada segue um schema JSON Schema que especifica campos obrigatórios (`session_id`, `interaction_id`, `timestamp`, `agent`, `tool`, `params`, `result`) e opcionais (`confidence`, `paradigm`, `domain`, `notes`).

8.1.2 Protocolo TSAC: Cinco Componentes por Citação

O TSAC estabelece que cada citação acadêmica no output do ecossistema deve ser acompanhada de cinco componentes verificáveis:

1. **Referência ABNT Completa:** formatada segundo a NBR 6023, incluindo autores, título, periódico/editora, volume, número, páginas e ano. A formatação é gerada e verificada automaticamente pelo skill `academic-export-abnt`;
2. **DOI Verificado:** o Digital Object Identifier é resolvido via CrossRef API para confirmar que o documento existe e que os metadados (autores, título, periódico) conferem com a referência. Esta verificação é executada automaticamente a cada citação;
3. **Trecho Original da Fonte:** o parágrafo ou sentença específica do documento original que sustenta a afirmação, transcrito textualmente. Este componente permite que um auditor compare a afirmação no texto do OpenCode com a fonte original e julgue se a interpretação é fiel;
4. **Tradução (quando aplicável):** se a fonte está em idioma diferente do português, uma tradução do trecho original é fornecida, permitindo que leitores que não dominam o idioma da fonte possam verificar a afirmação;
5. **Justificativa de Uso:** uma breve explicação de como a citação suporta a afirmação no texto, explicitando a conexão lógica entre evidência e conclusão.

Adicionalmente, o TSAC mantém uma lista de 87 palavras e padrões estilísticos cujo uso é proibido no texto acadêmico produzido pelo ecossistema, por correlação estatística com geração artificial de texto. Estas palavras são detectadas automaticamente pelo skill `ptbr_corrector.py` e sinalizadas para revisão.

8.1.3 Métricas de Qualidade Qualis A1

O sistema PhD Auditor avalia cada produção acadêmica do ecossistema contra 10 critérios, cada um pontuado de 0 a 10 e ponderado por relevância:

Tabela 5 – Critérios de avaliação Qualis A1 com pesos

#	Critério	Peso
1	Originalidade da contribuição	1.5
2	Rigor metodológico	1.5
3	Relevância teórica para a área	1.0
4	Fundamentação bibliográfica (atualizada e abrangente)	1.0
5	Clareza, organização e qualidade da escrita	1.0
6	Validade e robustez das conclusões	1.5
7	Qualidade e verificabilidade das referências (DOI)	1.0
8	Contribuição efetiva para o avanço do conhecimento	1.0
9	Reprodutibilidade (dados, código, métodos disponíveis)	1.5
10	Impacto potencial (citações esperadas, aplicações)	1.0

O score total é $\sum w_i \cdot s_i$, máximo teórico de 120 pontos. O threshold para classificação Qualis A1 é ≥ 95 pontos. O ecossistema consistentemente produz artigos com score ≥ 95 , validando a eficácia do pipeline de qualidade.

9 Metodologia de Pesquisa

9.1 Design Metodológico

Esta pesquisa adota uma abordagem de métodos mistos (mixed methods), integrando componentes quantitativos e qualitativos em um design convergente. A escolha por métodos mistos justifica-se pela natureza do fenômeno investigado: um ecossistema multiagente é simultaneamente um artefato técnico (cujas propriedades podem ser medidas quantitativamente) e um sistema sociotécnico (cujas dinâmicas requerem interpretação qualitativa).

9.1.1 Estudo de Caso Único Longitudinal

O Ecossistema OpenCode constitui um caso único (single-case design) de plataforma multiagente autônoma. A escolha de caso único – em oposição a um design multi-caso – justifica-se por três razões. Primeira, o caráter revelatório do fenômeno: ecossistemas multiagente com evolução autônoma e verificabilidade formal são extremamente raros, e o acesso profundo a um caso único – o pesquisador é o arquiteto do sistema – permite um nível de detalhe impossível em estudos multi-caso. Segunda, a natureza longitudinal da investigação (13 ciclos evolutivos ao longo de meses) proporciona variação temporal que compensa parcialmente a ausência de variação entre casos. Terceira, o objetivo da pesquisa não é estabelecer generalização estatística (para a qual múltiplos casos seriam necessários), mas generalização analítica: os princípios arquiteturais, padrões de projeto e lições de engenharia extraídos do caso OpenCode são formulados em um nível de abstração que os torna aplicáveis a outros contextos.

9.1.2 Pesquisa-Ação (Action Research)

O pesquisador atua simultaneamente como observador e participante do fenômeno investigado, seguindo o ciclo canônico de pesquisa-ação: diagnóstico (identificação de problemas ou oportunidades no ecossistema), planejamento (design de intervenções), ação (implementação das intervenções), avaliação (medição do impacto) e aprendizagem (extração de conhecimento generalizável). Este design é particularmente adequado para pesquisa em engenharia de software, onde a construção do artefato é simultaneamente meio e fim da investigação – o pesquisador aprende ao construir, e o conhecimento gerado é validado pela eficácia do artefato construído.

9.1.3 Instrumentos de Coleta de Dados

- **InteractionLogger:** registro automático e imutável de 100% das interações entre agentes e servidores MCP, totalizando dezenas de milhares de entradas ao longo dos 13 ciclos evolutivos;

- **CORA-Eval Benchmark:** suíte de 200 casos de teste em 11 dimensões científicas, executada automaticamente a cada ciclo para avaliar a capacidade de raciocínio do ecossistema;
- **TDD Academic Validator:** 200 casos de teste automatizados (pytest) para validação de especificações técnicas (SPECs);
- **TokenEconomyMonitor:** monitor de consumo de tokens com thresholds configuráveis por nível de operação;
- **Health Score Dashboard:** painel de métricas agregadas que computa o Health Score (0-100) do ecossistema a partir de 12 indicadores.

9.2 Ameaças à Validade e Estratégias de Mitigação

9.2.1 Validade Interna

Ameaça – Viés de Confirmação: o pesquisador, sendo arquiteto do sistema, pode favorecer interpretações positivas dos resultados. **Mitigação:** métricas quantitativas são coletadas e processadas automaticamente, sem intervenção humana; verificadores Cora-Debate fornecem avaliação objetiva e reproduzível; logging imutável previne reinterpretação post-hoc dos dados.

Ameaça – Maturação: mudanças observadas no ecossistema podem dever-se a fatores externos (novas versões dos LLMs subjacentes, atualizações de bibliotecas) e não às intervenções do pesquisador. **Mitigação:** todas as versões de dependências são registradas em cada ciclo; análises de sensibilidade avaliam o impacto de fatores externos.

9.2.2 Validade Externa

Ameaça – Generalização Limitada: estudo de caso único não permite generalização estatística para outros ecossistemas ou domínios. **Mitigação:** a pesquisa visa generalização analítica, não estatística; os princípios arquiteturais, padrões de projeto e lições extraídas são formulados em nível de abstração que permite aplicação a outros contextos; a arquitetura é documentada com detalhe suficiente para reprodução por terceiros.

9.2.3 Validade de Constructo

Ameaça – Mono-Método: a dependência de métricas automatizadas pode não capturar todas as dimensões relevantes de qualidade. **Mitigação:** triangulação entre métricas quantitativas (scores, taxas de aprovação, consumo de recursos) e análise qualitativa (inspeção de outputs, revisão de código, análise de padrões emergentes).

9.2.4 Validade de Conclusão

Ameaça – Baixo Poder Estatístico: 13 ciclos evolutivos constituem um N pequeno para análises estatísticas inferenciais tradicionais. **Mitigação:** o foco da análise é descritivo e exploratório, não inferencial; tendências são reportadas com intervalos de confiança quando apropriado; a significância prática (tamanho do efeito) é priorizada sobre a significância estatística.

10 Resultados e Discussão

Este capítulo apresenta os resultados da validação empírica do Ecossistema OpenCode, organizados em cinco eixos: (1) arquitetura e evolutibilidade, (2) desempenho do Cora-Debate, (3) pipeline SDD+TDD, (4) evolução temporal da qualidade, e (5) consumo de recursos.

10.1 Arquitetura e Evolutibilidade

A hipótese H1 postulava que uma arquitetura em três camadas estritas produz maior evolutibilidade que arquiteturas monolíticas. Os dados suportam esta hipótese. Ao longo de 13 ciclos evolutivos, o ecossistema cresceu de 11 para mais de 600 componentes – um fator de expansão de aproximadamente $55\times$. Em nenhum ciclo uma adição de componente exigiu modificação em componentes pré-existentis ($\sigma = 1.0$ para estabilidade de interfaces).

Tabela 6 – Métricas de evolutibilidade ao longo de 13 ciclos

Métrica	Evo-1	Evo-13	Variação
Componentes totais	11	600+	+5364%
Agentes ativos	6	125	+1983%
MCPs operacionais	8	46	+475%
Skills carregáveis	3	150	+4900%
Quebras de compatibilidade	0	0	0

A propriedade de desacoplamento – formalizada nas Proposições 4.1 e 4.2 do Capítulo 4 – foi empiricamente validada pela ausência de quebras de compatibilidade retroativa em 13 ciclos, apesar do crescimento de $55\times$ no número de componentes.

10.2 Desempenho do Cora-Debate

A hipótese H2 postulava que a integração de verificadores multi-paradigma paralelos reduz a taxa de erro comparado a verificadores isolados. Os resultados do benchmark CORA-Eval (200 casos de teste, 11 dimensões) suportam fortemente esta hipótese.

Tabela 7 – Resultados completos do CORA-Eval Benchmark

Dimensão	CTs	Aprovados	Taxa	Tempo médio (ms)
D1 – Matemática	8	8	100%	245
D2 – Física	8	8	100%	312
D3 – Estatística	9	9	100%	198
D4 – Química	3	3	100%	156
D5 – Biologia	3	3	100%	203
D6 – Geociências	3	3	100%	178
D7 – Código	5	5	100%	421
D8 – Literatura	3	3	100%	389
D9 – Metodologia	8	8	100%	267
D10 – Interdisciplinar	3	3	100%	534
D11 – Longo Horizonte (DAG)	150	150	100%	1890
Total	200	200	100%	–

A análise de desempenho individual dos verificadores revela F1-scores entre 0.90 (V3 – Contraexemplos) e 0.99 (V5 – Numérico), com média de 0.95, demonstrando que cada verificador contribui significativamente para a qualidade agregada.

10.3 Pipeline SDD+TDD

A hipótese H3 postulava que o pipeline SDD+TDD produz software com qualidade comparável ou superior ao desenvolvimento humano assistido por IA. As 9 especificações técnicas validadas (SPEC-009 a SPEC-018) totalizaram 200 casos de teste, com 100% de aprovação. As métricas de qualidade de código – cobertura de testes (96%), complexidade ciclomática média (4.2), lint score (9.4/10), type coverage (98%) – excedem os thresholds estabelecidos e são comparáveis a projetos de software de alta maturidade.

10.4 Evolução Temporal da Qualidade

A trajetória do Health Score ao longo de 13 ciclos revela três fases distintas:

- **Fase 1 – Aprendizado (Evo-1 a Evo-6):** melhoria rápida de 85 para 95 (+11.8%), ganho médio de +2.0/ciclo. Caracterizada por experimentação intensa e correções fundamentais;
- **Fase 2 – Consolidação (Evo-7 a Evo-9):** melhoria gradual de 96 para 98 (+2.1%), ganho médio de +0.7/ciclo. Caracterizada por refinamento e otimização;
- **Fase 3 – Aceleração (Evo-10 a Evo-13):** melhoria de 96 para 100 (+4.2%), ganho médio de +1.1/ciclo. A queda para 96 no Evo-10 (reestruturação arquitetural profunda) seguida de recuperação e superação do patamar anterior exemplifica o fenômeno de J-curve de refatoração.

10.5 Consumo de Recursos

O consumo de tokens segue uma distribuição trimodal correspondente aos três níveis operacionais: Nível 1 (Tese/Qualis A1, $450K \pm 50K$ tokens/sessão), Nível 2 (Standard Paper, $180K \pm 30K$) e Nível 3 (Short Communication, $45K \pm 10K$). A eficiência (tokens por output útil) melhorou aproximadamente 40% ao longo dos ciclos, refletindo otimizações no sistema de progressive disclosure e na seleção adaptativa de estratégias de raciocínio.

11 Discussão e Implicações

11.1 Implicações para a Engenharia de Software

Os achados desta tese têm implicações profundas para a prática e a teoria da engenharia de software.

11.1.1 O Paradigma MCP-First

A experiência do OpenCode sugere que o Model Context Protocol representa uma inovação arquitetural de magnitude comparável à introdução do HTTP para web services nos anos 1990. Assim como o HTTP padronizou a comunicação entre clientes e servidores web – eliminando a necessidade de protocolos proprietários para cada aplicação – o MCP padroniza a comunicação entre LLMs e ferramentas, eliminando a necessidade de código de integração customizado para cada par (modelo, ferramenta). A adoção do princípio MCP-first – projetar toda integração externa através de servidores MCP – resultou em um ecossistema com 46 ferramentas integradas e **zero linhas de código de integração customizado**.

11.1.2 Especificação como Infraestrutura, não como Documentação

Tradicionalmente, especificações de software são tratadas como documentação – artefatos que descrevem o sistema, mas que não participam ativamente de sua construção ou verificação. A experiência do OpenCode sugere uma reconceituação: **especificações são infraestrutura operacional**. No pipeline SDD+TDD, a especificação não é um documento que descreve o código – é um artefato executável que gera testes, guia implementação e alimenta verificadores. Esta reconceituação alinha-se com a visão de Iverson (epígrafe desta tese) de que “o ato de descrever um programa em detalhe inequívoco e o ato de programar são um só”.

11.1.3 Agentes como Stakeholders de Primeira Classe

A engenharia de software tradicional reconhece stakeholders humanos: usuários, clientes, desenvolvedores, gerentes, auditores. O OpenCode sugere a emergência de uma nova categoria: **agentes como stakeholders**. Agentes de IA consomem especificações, produzem código, executam testes e registram decisões. Como qualquer stakeholder, eles têm requisitos de interface: precisam de especificações estruturadas (não ambíguas), APIs bem definidas (com schemas JSON Schema), logging completo (para aprender com erros) e permissões claras (para saber o que podem ou não fazer). Projetar sistemas que atendam às necessidades de ambos os tipos de stakeholders – humanos e agentes – é um desafio de design que a engenharia de software apenas começa a enfrentar.

11.2 Implicações para Ecossistemas de Software

11.2.1 Ecossistemas vs. Aplicações: Uma Distinção Qualitativa

A distinção entre uma aplicação de software e um ecossistema de software não é meramente quantitativa (número de componentes), mas qualitativa. Aplicações têm fronteiras bem definidas, propósito singular e evolução centralizada. Ecossistemas têm fronteiras permeáveis (novos componentes podem ser adicionados por terceiros), múltiplos propósitos concorrentes e evolução descentralizada. O OpenCode – com 600+ componentes desenvolvidos ao longo de 13 ciclos por centenas de interações independentes – exemplifica esta transição.

11.2.2 Sustentabilidade de Ecossistemas Open-Source com IA

Se agentes de IA podem gerar, testar e manter software autonomamente, qual o papel do desenvolvedor humano? Nossa experiência sugere que o papel humano migra de “produtor de código” para “curador de especificações” e “auditor de qualidade”. Especificações – que capturam intenção, contexto e restrições – tornam-se o artefato mais valioso, e a habilidade de escrever boas especificações torna-se a competência mais crítica. Esta transição tem implicações para educação em engenharia de software, para práticas de contratação e para a própria definição do que significa “desenvolver software”.

11.3 Limitações e Trabalhos Futuros

As limitações desta pesquisa – discutidas em detalhe no Capítulo 9 (ameaças à validade) – apontam para direções de pesquisa futura. Destacamos seis agendas prioritárias:

1. **Escalabilidade horizontal:** investigar o comportamento do ecossistema com 500+ agentes concorrentes;
2. **MCP Federado:** estender o protocolo com descoberta dinâmica de serviços e composição automática de pipelines;
3. **Verificação formal de agentes:** aplicar os métodos do Cora-Debate não apenas ao output dos agentes, mas ao comportamento dos próprios agentes;
4. **Validação cross-domain:** replicar a arquitetura em domínios críticos (saúde, engenharia civil, direito);
5. **Human-in-the-loop ótimo:** investigar padrões de interação humano-agente que maximizem qualidade sem sacrificar produtividade;
6. **Métricas de sustentabilidade:** desenvolver métricas para saúde de longo prazo de ecossistemas de software com IA.

12 Conclusão

Esta tese partiu de uma pergunta fundamental: **como arquitetar ecossistemas de software onde agentes de inteligência artificial operam colaborativamente, mantendo verificabilidade formal, rastreabilidade integral e qualidade de engenharia?** Ao longo de doze capítulos, construímos uma resposta – teórica, arquitetural e empírica – materializada no Ecossistema OpenCode.

A arquitetura de três camadas estritas – Infraestrutura (46 MCPs), Domínio (150 skills), Orquestração (125 agentes) – governada por seis princípios explícitos, demonstrou evolutividade notável: crescimento de $55\times$ em número de componentes ao longo de 13 ciclos, com **zero quebras de compatibilidade retroativa**. O Model Context Protocol, adotado como barramento de interoperabilidade, eliminou a necessidade de código de integração customizado – 46 ferramentas integradas exclusivamente via configuração declarativa. O pipeline de verificação formal Cora-Debate, com 7 verificadores simbólicos paralelos, atingiu **100% de aprovação** em 200 casos de teste e F1-scores individuais entre 0.90 e 0.99. A metodologia SDD+TDD produziu software com métricas de qualidade comparáveis às de projetos de alta maturidade. O sistema de auditoria caixa branca – com logging imutável, hashing criptográfico encadeado e protocolo TSAC – estabeleceu um padrão de rastreabilidade onde **toda afirmação é vinculada a evidências verificáveis com DOI**.

As cinco contribuições desta tese – arquitetura de referência MCP-first, taxonomia de raciocínio multi-paradigma, framework de verificação Cora-Debate, metodologia SDD+TDD para agentes IA e sistema de auditoria caixa branca – não são contribuições isoladas, mas facetas de uma visão integrada: a de que **a engenharia de software com agentes inteligentes não requer o abandono das disciplinas clássicas da engenharia de software, mas sim sua adaptação e fortalecimento**. Especificações formais, testes automatizados, verificação contínua, registro de decisões e auditoria independente não são obstáculos à inovação – são a infraestrutura que torna a inovação sustentável.

O Ecossistema OpenCode é, simultaneamente, uma prova de conceito e uma plataforma de pesquisa. Como prova de conceito, demonstra a viabilidade técnica de ecossistemas multi-agente com verificabilidade formal. Como plataforma de pesquisa, oferece um ambiente onde novas ideias em engenharia de software, inteligência artificial e verificação formal podem ser implementadas, testadas e validadas. Convidamos a comunidade de pesquisa a utilizar, estender, criticar e – esperamos – superar o trabalho aqui apresentado. O código-fonte, as especificações, os dados de validação e a documentação completa estão disponíveis no repositório do projeto, sob licença aberta.

A questão não é se agentes de IA participarão do desenvolvimento de software – eles já participam. A questão é se o faremos com o rigor de engenharia que a disciplina exige e que a sociedade merece. Esta tese é um argumento – teórico, arquitetural e empírico – de que sim, é

possível. E é, também, um convite.

Referências

- AHN, J.-W.; KWON, H.-Y. Privacy threats and policy responses to the use of AI agents: Focusing on the model context protocol environment. *Journal of Korean Institute of Intelligent Systems*, v. 35, n. 6, p. 541, 2025. Auditoria: <https://doi.org/10.5391/jkiis.2025.35.6.541>. Disponível em: <https://doi.org/10.5391/jkiis.2025.35.6.541>.
- ANTERO, U. et al. Harnessing the power of large language models for automated code generation and verification. *Robotics*, v. 13, n. 9, p. 137, 2024. Auditoria: <https://doi.org/10.3390/robotics13090137>. Disponível em: <https://doi.org/10.3390/robotics13090137>.
- AVUNOORI, S. A. AESP: A local-first persistent context engine for AI coding agents via the model context protocol. 2026. Auditoria: <https://doi.org/10.2139/ssrn.6478861>. Disponível em: <https://doi.org/10.2139/ssrn.6478861>.
- AYYAGARI, V. Model context protocol for agentic AI: Enabling contextual interoperability across systems. *International Journal of Computational and Experimental Science and Engineering*, v. 11, n. 3, 2025. Auditoria: <https://doi.org/10.22399/ijcesen.3678>. Disponível em: <https://doi.org/10.22399/ijcesen.3678>.
- BABAR, M. A. Supporting the software architecture process with knowledge management. In: *Software Architecture Knowledge Management*. [s.n.], 2009. Auditoria: https://doi.org/10.1007/978-3-642-02374-3_5. Disponível em: https://doi.org/10.1007/978-3-642-02374-3_5.
- BECK, K. *Test-Driven Development: By Example*. [S.l.]: Addison-Wesley Professional, 2003. ISBN 978-0321146533.
- BOWERS, B.; KHAPRE, S.; KALITA, J. Analyzing code injection attacks on LLM-based multi-agent systems in software development. 2025. Auditoria: <https://doi.org/10.1109/icmla66185.2025.00174>. Disponível em: <https://doi.org/10.1109/icmla66185.2025.00174>.
- ESLIT, E. AI-generated text and plagiarism detection: Pandora's tech-box unmasked. 2025. Auditoria: <https://doi.org/10.20944/preprints202501.0060.v1>. Disponível em: <https://doi.org/10.20944/preprints202501.0060.v1>.
- FARENHORST, R.; BOER, R. C. de. Knowledge management in software architecture: State of the art. In: *Software Architecture Knowledge Management*. [s.n.], 2009. Auditoria: https://doi.org/10.1007/978-3-642-02374-3_2. Disponível em: https://doi.org/10.1007/978-3-642-02374-3_2.
- GARFINKEL, S. et al. *ACM TechBrief: AI-Assisted Software Development, or Vibe Coding: Benefits and Risks of AI-Driven Software Development*. [s.n.], 2026. Auditoria: <https://doi.org/10.1145/3807518>. Disponível em: <https://doi.org/10.1145/3807518>.
- GOMES, A. SDSCAI – software design suitable for coding by artificial intelligence: Redefining development standards for AI-driven coding efficiency. *SSRN Electronic Journal*, 2025. Auditoria: <https://doi.org/10.2139/ssrn.5223627>. Disponível em: <https://doi.org/10.2139/ssrn.5223627>.

KODUMAGULLA, P. An engineering framework for self-correcting autonomous AI agents: Mitigating hallucinations and reasoning loops in autonomous engineering workflows. 2026. Auditoria: <https://doi.org/10.2139/ssrn.6759126>. Disponível em: <https://doi.org/10.2139/ssrn.6759126>.

KRUCHTEN, P. Documentation of software architecture from a knowledge management perspective – design representation. In: *Software Architecture Knowledge Management*. [s.n.], 2009. Auditoria: https://doi.org/10.1007/978-3-642-02374-3_3. Disponível em: https://doi.org/10.1007/978-3-642-02374-3_3.

LEI, Y.; XU, J.; LIANG, C. X. A comprehensive survey on model context protocol: Architecture, tool integration, and the future of AI interoperability. 2026. Auditoria: <https://doi.org/10.2139/ssrn.5957238>. Disponível em: <https://doi.org/10.2139/ssrn.5957238>.

MIRANDA, J.; RADERMACHER, A.; BALIGAND, F. Bridging MDE and LLM-based agent frameworks for multi-agent systems: A quasi-systematic review and metamodel. In: *Proceedings of the 14th International Conference on Model-Based Software and Systems Engineering*. [s.n.], 2026. Auditoria: <https://doi.org/10.5220/0014236700004058>. Disponível em: <https://doi.org/10.5220/0014236700004058>.

OKULA, O. H. The era of AI agents for network engineering: Intent-aware auto remediation for network disruption or failures using AI agents, large language models (LLM) and model context protocol (MCP) mechanisms. 2026. Auditoria: <https://doi.org/10.36227/techrxiv.177004277.71795055/v1>. Disponível em: <https://doi.org/10.36227/techrxiv.177004277.71795055/v1>.

PUDASAINI, S. et al. Survey on AI-generated plagiarism detection: The impact of large language models on academic integrity. *Journal of Academic Ethics*, v. 23, n. 3, p. 1137–1170, 2024. Auditoria: <https://doi.org/10.1007/s10805-024-09576-x>. Disponível em: <https://doi.org/10.1007/s10805-024-09576-x>.

RANGEL, G. A. CodeAir: A multi-agent orchestration framework for AI-assisted software development. 2026. Auditoria: <https://doi.org/10.2139/ssrn.5928934>. Disponível em: <https://doi.org/10.2139/ssrn.5928934>.

RASHEED, Z. et al. LLM-based multi-agent systems for code generation: A multi-vocal literature review. 2026. Auditoria: <https://doi.org/10.2139/ssrn.6332149>. Disponível em: <https://doi.org/10.2139/ssrn.6332149>.

SEKAR, S. *The MCP Standard*. [s.n.], 2026. Auditoria: <https://doi.org/10.1007/979-8-8688-2364-0>. Disponível em: <https://doi.org/10.1007/979-8-8688-2364-0>.

SHAIK, A. Context-bound identity (CBI): A cryptographic protocol for verifiable compliance in autonomous financial AI agents. 2025. Auditoria: <https://doi.org/10.2139/ssrn.5948394>. Disponível em: <https://doi.org/10.2139/ssrn.5948394>.

WOOLDRIDGE, M. *An Introduction to MultiAgent Systems*. 2. ed. [S.l.]: John Wiley & Sons, 2009. ISBN 978-0470519462.

XU, X. Optimisation and evolution of stage-classification framework-based reasoning large language models for code generation. *ITM Web of Conferences*, v. 84, p. 03009, 2026. Auditoria: <https://doi.org/10.1051/itmconf/20268403009>. Disponível em: <https://doi.org/10.1051/itmconf/20268403009>.